MEDICAL AI · FEDERATED LEARNING · BLOCKCHAIN

MedXChAIn

Technical Audit & Enhancement Roadmap

A complete engineering review of a decentralized, privacy-preserving federated learning system for distributed hospitals — followed by a build-ready plan to make it real.

Repository	SuhasRam356/Decentralized-Privacy-Preserving-Federated-Learning-...-Hospitals
Baseline commit	b0b8941044e807ff0ee628ad66ab9468d94ea1bd (single commit, 2026-10-22)
Branch audited	arena/144b2365-decentralized-privacy-preservi
Method	Full source read · static analysis · live execution of contracts (solc 0.8.28 + Hardhat), the IPFS emulator (4 ports), and every dashboard API endpoint
Scope	66 tracked files · 137 MB working tree · 573 LOC Python · 510 LOC Solidity · 3 638 LOC frontend · 14 contract tests
Prepared for	An implementing agent (human or AI) — see Part 0 for the working contract
Companion files	TECHNICAL_AUDIT.md · ENHANCEMENT_ROADMAP.md · agent_backlog.yaml

Part	Contents	Use it for
0	Agent briefing — mission, 8 non-negotiable rules, gates, verification commands, 8 landmines found by execution, definition of done, reporting format	Hand this to the implementing agent first
I	Technical audit — verdict, 19-row documentation-drift matrix, 27 findings (C/H/M/L) with reproducible evidence, live-execution transcripts, privacy & legal review	Understand what is wrong and why it matters
II	Enhancement roadmap — 12 quick wins, 11 tracks, target architecture, 18 new features, SLOs, security, compliance, first 10 PRs, what not to build	Plan and execute the rebuild
A	Backlog summary — 18 tasks with priority, findings closed, effort and dependencies (full machine-readable version in agent_backlog.yaml)	Track progress task by task

Headline verdict: an impressive demonstration shell around a non-functional scientific core. The UI, contract layout, CID math and documentation are real work; the federated learning, on-chain coordination, IPFS integration and clinical inference are simulated, stubbed or disconnected — and 19 documented claims contradict the code.

Contents

Part 0 — Agent Briefing (read before touching code) . . . 4

0.1 Mission . . . 4
0.2 Non-negotiable rules . . . 4
0.3 Order of work and gates . . . 5
0.4 Verification commands . . . 5
0.5 Landmines found by execution — read these before debugging . . . 5
0.6 Definition of done — per task . . . 6
0.7 Reporting format — what to send back after each PR . . . 6
0.8 Ask before doing (stop conditions) . . . 6
0.9 Explicitly out of scope for this engagement . . . 7
0.10 The one-sentence summary of the job . . . 7

Part I — Technical Audit . . . 7

0. Verdict at a glance . . . 7
1. What the project actually is . . . 8
2. Inventory (measured) . . . 8
3. Component deep dives . . . 9
4. Runtime verification (what was actually executed) . . . 13
5. Findings register . . . 13
6. Documentation drift matrix . . . 15
7. Privacy, safety and legal . . . 16
8. Performance & scalability observations . . . 17
9. What is genuinely good . . . 17
10. Prioritized remediation plan . . . 17
Appendix A — Reproducible command log . . . 18
Appendix B — Artefacts produced with this audit . . . 19

Part II — Enhancement & New-Feature Plan . . . 20

0. How to read this . . . 20
1. The 12 quick wins (start today, ~8 pd total) . . . 20
2. North-star architecture . . . 20
3. Target repository layout . . . 21
4. Phase plan at a glance . . . 22
5. Track A — Make the federated learning real . . . 22
6. Track B — Privacy engineering (the title finally becomes true) . . . 26
7. Track C — Blockchain v2 (FederatedRound.sol) . . . 27
8. Track D — Storage v2 . . . 30
9. Track E — Orchestration, API, and the database . . . 31

10. Track F — Dashboard 2.0 . . . 32
11. Track G — Clinical AI that is safe to show a doctor . . . 34
12. Track H — 18 new features . . . 35
13. Track I — Security engineering . . . 38
14. Track J — Compliance & governance . . . 38
15. Track K — MLOps, CI/CD, observability, deployment . . . 39
16. Targets & SLOs . . . 39
17. Finding → work-item coverage . . . 40
18. The first 10 pull requests (copy-paste backlog) . . . 40
19. What NOT to build (yet) . . . 41
20. Definition of done (per track) . . . 41
21. Risks & trade-offs . . . 41
22. Appendix A — environment variables . . . 42
Appendix B — database schema (essentials) . . . 42
Appendix C — dependency additions . . . 42
23. TL;DR . . . 43

Appendix A — Task backlog summary . . . 44

Part 0 — Agent Briefing (read before touching code)

This briefing is addressed to the engineering agent (human or AI) that will implement the work. Parts I and II are the evidence and the plan; this is the contract for the engagement.

0.1 Mission

Take this repository from **"impressive demo shell around a non-functional core"** to **"verified prototype"**: one real federated round, every displayed number traceable to a source, and no claim in the documentation that the code does not support.

Success is measured, not asserted. When this engagement is done, all five statements below must be true and demonstrable in under five minutes:

1. `make round` executes **train → encrypt → IPFS → submit → aggregate → publish → evaluate** across three local sites and exits 0.
2. Every figure the dashboard renders can be reproduced with a single SQL query against Postgres.
3. `forge test` passes at ≥ 95 % coverage and `pytest` at ≥ 70 %, and CI is green on `main`.
4. The README contains no claim that is not backed by a link to a metrics file, a test or a compliance artefact.
5. The regression greps in `agent_backlog.yaml → regression_guards` return nothing.

0.2 Non-negotiable rules

Rule	Why it exists	How it is checked
R1 — Never fabricate a value. No invented CID, hash, transaction, accuracy, timestamp or "system status".	The current codebase does exactly this (findings C-02, C-05, H-02) and it is the single reason the project cannot be trusted.	`grep -rn 'QmHospital|Simulated|deadbeef|1\.2k|89\.4'` must be empty
R2 — Fail closed. A missing dependency raises; it never degrades to a placeholder.	`try/except: use fake CID` is the pattern that created the problem.	Code review + a test per failure path
R3 — Every number has provenance. API responses carry `sources: {chain, ipfs, model, db}` and a `generated_at`.	Lets the UI show "degraded" instead of lying.	`tests/api/test_status.py` asserts provenance fields
R4 — No compliance claims. Write "designed against HIPAA §164.312"; never "HIPAA compliant".	An audit has not happened; overclaiming is the repo's existing failure mode.	`grep -rni 'hipaa compliant|gdpr certified'` must be empty
R5 — Tests travel with code. A PR without a test for its new behaviour is incomplete.	There are currently zero Python tests.	CI coverage gate
R6 — Preserve the user's data. Do not delete or rewrite `FL/Train&TestCSVs/` or `LocalTraining/` without explicit approval; T-04/T-05 require a dry-run report first.	Irreversible; the data may be the only copy.	Stop condition, see §0.8
R7 — Small PRs. ≤ 600 changed lines, one concern, conventional commit message.	Reviewability, and it makes each step independently revertible.	PR template checklist

Rule	Why it exists	How it is checked
R8 — Pin dependencies. No silent upgrades of `torch`, `transformers`, `opacus`, `web3`.	Reproducibility is a stated research goal.	Lock files committed

0.3 Order of work and gates

Do the tasks in `agent_backlog.yaml` in ID order. The gates below are hard: do not start the next block before the previous one is green, because each block's tests are the safety net for the next.

Gate	Tasks	Exit test	Why the gate exists
G0 — Honesty	T-01 … T-07	`grep` guards empty, CI green	Every later PR would otherwise add features on top of fabricated output, making the lies harder to find
G1 — Real training	T-08 … T-10	`tests/fl` green, determinism test passes	An aggregator over random models (today) cannot be evaluated, so nothing downstream can be validated
G2 — Chain + service	T-11 … T-14	`make round` exits 0, dashboard shows only DB values	This is the first point at which the system is end-to-end real
G3 — Clinical	T-15	`tests/clinical` green, no keyword matching left	Requires a real checkpoint (G1) and a real API (G2)
G4 — Privacy	T-16, T-17	ε ledger populated, artefacts unreadable without keys	DP over a full 110 M-parameter model is not runnable; it depends on LoRA (T-09)
G5 — Auditability	T-18	`round replay N --verify` passes	Needs real transactions and a real DB to replay

0.4 Verification commands

Run these before every push. The environment notes matter: in a sandbox without outbound access to `binaries.soliditylang.org`, Hardhat cannot download `solc`.

```
# Python
pytest -q --cov=medxchain --cov-fail-under=70
ruff check . && ruff format --check . && mypy src

# Contracts — preferred
cd contracts && forge fmt --check && forge test -vvv && forge coverage --report summary

# Contracts — legacy Hardhat path (if forge is unavailable)
cd Blockchain && npx hardhat test --no-compile

# Contracts — compile when the compiler cannot be downloaded:
#   npm install --no-save solc@0.8.28
#   then compile contracts/ with the npm solc package (optimizer enabled, runs=200)
#   and emit Hardhat-format artefacts to Blockchain/artifacts/contracts/**.json
#   (this is exactly how the audit compiled them: 5 artefacts, 0 errors, 0 warnings)

# Regression guards
grep -rn 'QmHospital\|Simulated' --include=*.py .
grep -rn '0x5FbDB2315678afecb367f032d93F642f64180aa3' --include=*.py --include=*.js .
grep -rn 'e:/Fedacre\|/content/drive' --include=*.py .

# Behaviour
python -m medxchain.cli round run --sites 3 --verify-chain
python -m medxchain.cli round replay 1 --verify
```

0.5 Landmines found by execution — read these before debugging

These were discovered by running the code, not by reading it. Each will cost you hours if you hit it blind.

1. **The two halves of the ML system are incompatible.** The notebooks' model takes `(input_ids, attention_mask)` and its state dict has `encoder.*`, `classifier.0.*`, `classifier.3.*`.

`run_federated_pipeline.py` takes a flat 768-vector and has `feature_extractor.*`, `classifier.*`. `load_state_dict` fails in both directions. Fix with the single checkpoint format in T-09 — do not try to "convert" ad hoc.

2. **Kubo rejects the current upload.** `requests.post(url, data=content)` sends raw bytes; `/api/v0/add` requires `multipart/form-data`. Against a real daemon this fails 100 % of the time — which is why nobody noticed the fake CIDs.
3. **The IPFS emulator lies convincingly.** `path.endswith("/add")` also matches `/api/v0/pin/add`, so a "pin" call stores an empty file and returns the CID of the empty string. Unknown routes (`/swarm/peers`) return HTTP 200 with an empty body. Any health check that only looks at the status code passes against a broken node.
4. **All four emulated "nodes" share one storage directory.** A file added on 5001 is readable from 5004 — impossible in a real cluster. Do not cite this as replication.
5. **Hardhat's compiler download is blocked in restricted networks** (`HH502`). Use the npm `solc` package as described in §0.4; this is an environment issue, not a repo defect.
6. `deployment_info.json` **is committed, gitignored, and stale.** `.env.example` advertises a different address than the committed deployment, and the README's second address matches neither. Pick one source of truth (the deploy output) and load it through config.
7. `server.py` **imports** `torch` **and never uses it.** It costs ~2 s of startup and a 2 GB dependency. Remove it; do not "fix" it by adding torch usage.
8. `LocalTraining/` **is a byte-identical duplicate** of `FL/Train&TestCSVs/` (verified by md5). Deleting the wrong copy loses nothing, but confirm before acting (R6).

0.6 Definition of done — per task

A task is done only when **all** boxes are true:

- [ ] Code under `src/medxchain/` (or the specified location) with type hints; `mypy` clean on new modules
- [ ] Unit test for the happy path **and** for each failure path introduced
- [ ] The specific `verify` command from `agent_backlog.yaml` passes
- [ ] No new `TODO`, no `except: pass`, no placeholder returned on error (R1/R2)
- [ ] Metrics/log line added for the new failure mode, with an alert rule (F14) if the service can be degraded
- [ ] `docs/` updated; any README change links to the artefact that proves it (R4)
- [ ] Migration + rollback noted for DB or contract changes
- [ ] Manifest fields recorded so the round remains replayable (T-18)

0.7 Reporting format — what to send back after each PR

Keep it short and verifiable; this is what a reviewer can check without reading the diff.

```
TASK: T-09  Single checkpoint format + LoRA
STATUS: done | blocked | partial
CHANGED: src/medxchain/fl/model.py (+180/-12), tests/fl/test_checkpoint.py (new)
WHY: closes H-06 (incompatible checkpoints), enables H-08 fix via adapter-only averaging
EVIDENCE:
  pytest tests/fl/test_checkpoint.py -q        -> 7 passed
  ratio test (lora/full params)                -> 0.0027 (< 0.01 gate)
  grep -rn 'feature_extractor' src/            -> only model.py:31 (mapping helper)
BREAKING: checkpoint format v1; old .pt files cannot load (migration: re-train or convert via tools/convert_ckpt.py)
NEXT: T-10 (aggregator) — unblocked
OPEN QUESTION: none
```

0.8 Ask before doing (stop conditions)

Do not proceed silently on any of these; report and wait for approval:

- Deleting, rewriting or re-partitioning anything under `FL/Train&TestCSVs/` or `LocalTraining/` (R6)

- Rewriting git history (`filter-repo`, force-push) — required eventually for the PII scrub, but it is destructive and irreversible
- Adding a dependency that downloads weights or binaries at install time
- Changing the contract ABI after T-12 without regenerating `docs/abi.md`
- Enabling `MEDXCHAIN_DEMO_MODE=false` on anything that looks like production
- Any claim, diagram or README line that the implementation cannot demonstrate

0.9 Explicitly out of scope for this engagement

Tokenomics, on-chain model storage, zkML proofs, Kubernetes, microservice decomposition, a custom chain, and any framework rewrite for its own sake. See `ENHANCEMENT_ROADMAP.md` §19 for the reasoning — each of these adds cost and risk without moving the five success criteria in §0.1.

0.10 The one-sentence summary of the job

Replace every plausible-looking fabrication with a smaller, uglier, verifiable truth — then build features only on top of what can be proven.

Part I — Technical Audit

What is actually there, what it does when executed, and where the documentation diverges from the code.

Attribute	Detail
Repository	`SuhasRam356/Decentralized-Privacy-Preserving-Federated-Learning-with-Blockchain-for-Distributed-Hospitals`
Commit audited	`b0b8941` (single commit, 2026-10-22)
Branch	`arena/144b2365-decentralized-privacy-preservi`
Audit date	2026-10-22
Scope	every tracked file (66 files, ~137 MB working tree)
Method	full source read + static analysis + **live execution** of the Solidity contracts, the IPFS emulator, the FL pipeline components and the dashboard API (commands in Appendix A)

0. Verdict at a glance

Layer	Code maturity	Does it do what the README says?	Worst problem
Federated learning (`FL/`, `LocalTraining/`, `run_federated_pipeline.py`)	Prototype	✘ No	"Training" runs on `torch.randn` tensors — no patient data is ever learned
Smart contracts (`Blockchain/contracts/`)	Prototype, compiles, 14 tests pass	⚠ Partly	Accepts empty CID/hash as a valid submission; fully centralized admin
Blockchain integration (Python side)	**Missing**	✘ No	The pipeline **never sends a transaction**; `deployment_info.json` is only printed
IPFS layer (`ipfs-hospitals/`)	Working emulator	⚠ Partly	Not a real IPFS cluster (shared folder), and the client upload format is incompatible with Kubo
Dashboard (`UI-DashBoard/app/`)	Polished UI, hardcoded data	✘ No	Reports synthetic numbers (89.4 % accuracy, 14 hospitals, 1.2 k updates) as if measured

Layer	Code maturity	Does it do what the README says?	Worst problem
Clinical inference (`/api/diagnose`)	**Demo stub**	✘ No	Substring matching on disease names, not symptoms — produces medically wrong, confident output
Privacy / compliance	Claimed, absent	✘ No	No DP, no secure aggregation, no encryption — and raw patient-like PII is committed to Git
Engineering hygiene	Fair	⚠ Partly	No CI, no LICENSE, no Python tests, 134 MB of duplicated CSVs, stale `.pyc`

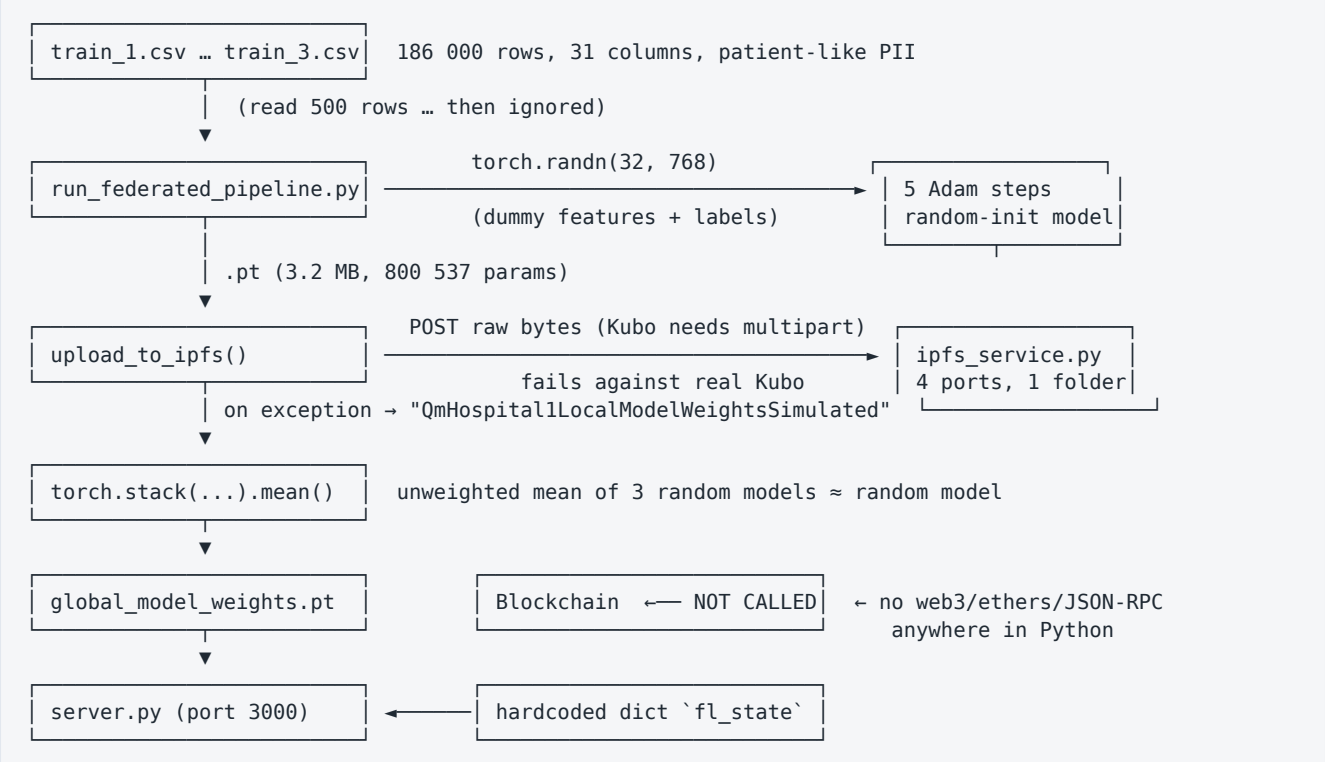
Bottom line: this is an **impressive-looking demonstration shell** around a **non-functional scientific core**. The UI, the contract layout, the CID math and the documentation are real work; the federated learning, the on-chain coordination, the IPFS integration and the clinical inference are **simulated, stubbed or disconnected**. Read as a prototype/proof-of-concept with a presentation layer, it is coherent. Read as the system the README describes (privacy-preserving clinical AI with an immutable on-chain audit trail), it is **not there yet**, and in several places it makes **claims that the code contradicts** (Section 7).

---

1. What the project actually is

A three-way proof of concept: three "hospital" notebooks train a MedCPT classifier, weights are averaged (FedAvg), CIDs are pushed to four IPFS daemons, and a Hardhat contract records round/slot metadata. A single-page dashboard visualises the network.

As implemented, the data path is:



Everything downstream of "IPFS" is disconnected from everything upstream. The chain never sees the pipeline; the dashboard never sees the chain, the files, or the model.

---

2. Inventory (measured)

Metric	Value
Tracked files	66
Git commits	**1** (`feat: complete MedXChAIn overhaul…`)
Disk size (with `.git`)	137 MB; unique CSV blobs 67 MB, stored **twice** (`FL/` + `LocalTraining/`, byte-identical)
Python	573 LOC across 4 files (pipeline 200, server 141, IPFS 163, addToIPFS 69)
Solidity	510 LOC across 4 contracts (ModelUpdateTracker 185, ModelRegistry2 193, ModelRegistry 100, Lock 32)
JavaScript	`app.js` 380 LOC + 5 Hardhat scripts (~292 LOC)
HTML / CSS	908 / 2 350 LOC, 6 views, single page, no framework
Tests	14 Hardhat tests (9 boilerplate `Lock.js`, 5 `ModelUpdateTracker`) — **0** for `ModelRegistry`/`ModelRegistry2`, **0** Python, **0** frontend
CI	none (`.github/` absent)
LICENSE	none, though README badge says MIT
Declared Python deps	10; **7 never imported** by any shipped script (`transformers`, `accelerate`, `numpy`, `scikit-learn`, `joblib`, `lime`, `tqdm` — the first three appear only inside the Colab notebooks); `web3` — required to actually talk to the chain — **is not declared**

---

3. Component deep dives

3.1 `run_federated_pipeline.py` — the "FL orchestrator"

What it does, in order:

1. Loads `FL/label_mapping.json` (793 classes) — genuine.
2. Reads `Blockchain/deployment_info.json` and **prints** the contract address (line 88-89). This is the only blockchain interaction in the entire Python codebase. There is no `web3`, no `eth_sendTransaction`, no `submitLocalModel` call, yet the script aborts with `FileNotFoundError` if the file is missing (line 69) — a hard dependency on a component it does not use.
3. For each "hospital": `pd.read_csv(..., nrows=500)` (line 98) → the DataFrame is **never used**. Training then runs 5 Adam steps on `torch.randn(32, 768)` and `torch.randint(...)` (lines 103-112). No seed is set, so every run produces different weights, and the "accuracy" is not computed at all.
4. Saves `state_dict` to `FL/LocalModels/hospital_N_weights.pt` and computes a real SHA-256 (correct implementation).
5. Uploads to `http://127.0.0.1:{5000+h}/api/v0/add` by POSTing **raw bytes** (`requests.post(url, data=content)`, line 52). Kubo's `/add` requires `multipart/form-data`. Against real Kubo this calls fails 100 % of the time.
6. On failure (i.e. always, with Docker), swallows the exception and invents a CID: `f"QmHospital{h_id}LocalModelWeightsSimulated"` (line 141). These strings are **not valid CIDv0** (37 chars, not a base58-encoded multihash) and are **content-independent**, so two different models get the same "CID". They are then displayed as if they were real.
7. FedAvg: `torch.stack(...).mean(dim=0)` (lines 149-151). **Uniform**, not sample-weighted, contradicting the README §4 formula; datasets are actually equal-sized so the result is mathematically the same — but the claim is still false.
8. Prints clinical differentials from `torch.randn(1, 768)` (line 181) — random logits presented as "confidence".
9. Final banner unconditionally prints `IPFS Decentralized CIDs Verified: Yes` and a **hard-coded** contract address (lines 194-196), regardless of what happened.

Structural incompatibility: the pipeline's `MedCPTClassifier` consumes a flat 768-d vector (keys: `feature_extractor.*`, `classifier.*`), while the notebooks' model consumes token IDs (keys: `encoder.*`, `classifier.0.*`, `classifier.3.*`). The two `state_dict`s **cannot be loaded into each other**. Any attempt

to combine the "real training" notebooks with this pipeline breaks on `load_state_dict`.

3.2 Notebooks

	`LocalTraining/LocalTrainingNotebook.ipynb`	`FL/GlobalModelTrainingNotebook.ipynb`
Domain	Real fine-tuning of `ncbi/MedCPT-Query-Encoder` on `symptoms` → 793 classes	Loading 3 checkpoints + averaging
Stored outputs	Test Accuracy **81.23 %** (cell 9) — but the 10-epoch training cell (8) has **no output**, so the run is incomplete/unverifiable	Global accuracy **73.84 % / 73.87 % / 73.97 %** per hospital test set
Problems	Colab `drive.mount` paths only; duplicates the model class in cells 5 and 6; always saves as `hospital_3_weights.pt` (hard-coded 3, so it cannot produce hospital 1/2 artefacts); no seeds; `execution_count` stripped	`NUM_LABELS = 797` (line 2164) contradicts the 793-class ontology (the variable is unused, so it is cosmetic but misleading); Turkish comments; no weighting, no aggregation diagnostics, no evaluation of the local vs global trade-off

The single most important number in the repo: the only real measurements available show the federated global model at **≈ 73.9 %**, i.e. **7.3 points worse** than the single local model (**81.23 %**). Uniformly averaging full encoder weights of separately fine-tuned MedCPT models is expected to degrade accuracy; nothing in the codebase addresses this (no FedProx, no layer-selective aggregation, no learning-rate/round schedule). Meanwhile the dashboard and README advertise **89.4 %** — a figure that exists nowhere in the data.

3.3 Dataset

- 31 columns per row: `Patient_First_Name`, `Patient_Last_Name`, `Patient_Date_Of_Birth`, `Patient_Email`, `Patient_Phone_Number`, `Patient_Insurance_Provider`, `Billing Amount`, emergency-contact name/phone/email, treating doctor's name, **plus** `symptoms`, `Disease_name`, `label`.
- Rows: 49 613 / 49 614 / 49 613 train + 12 387 / 12 386 / 12 387 test = **62 000 per hospital, 186 000 total**. (README says the train files have 62 000 rows each; they actually hold ~49.6 k train + 12.4 k test.)
- Data look synthetic (Faker-style e-mails such as `misty.richards1@hotmail.com`, negative phone numbers, DOB stored as Excel serial `33239`) — but the repository still ships **patient-shaped PII**, duplicated in two directories, in plain text, forever in Git history.
- Label distribution is long-tailed: 737 of 793 classes appear in `train_1`; **68 classes have exactly one example**; the largest class has 350. A single top-1 accuracy figure over this distribution is close to meaningless — macro-F1 / top-5 / per-class recall are all absent.
- Partitions are **IID random splits** (row sets are disjoint: 0 overlap of `(name, DOB, symptoms, label)` tuples across hospitals), so the hardest part of federated learning — non-IID heterogeneity — is never exercised.
- **7.2 % of `test_1` rows have an exact duplicate `symptoms` string in `train_1`** (889 / 12 387), so reported test accuracy is mildly optimistic.
- Labels are duplicated in both `Disease_name` and numeric `label`; training on the full CSV would leak the target.

3.4 IPFS layer

Two independent implementations exist:

`ipfs-hospitals/ipfs_service.py` — a 163-line Kubo API emulator. Verified live:

- CIDv0 math is **correct**: `sha256(data)` → multihash `0x12 0x20 …` → base58. My independent call returned `QmYjyGJrNaqN9RuKUjW3QKtRZNNe2n2aiHTan9r3Lx1R2L`, matching the server exactly.
- **Not distributed:** all four ports (`5001-5004`) write to the same `storage/` folder, so "cat from node 5004 of a file added on 5001" succeeds — physically impossible in a real cluster. It simulates the API, not the network.

- **Routing by suffix:** `path.endswith("/add")` also matches `/api/v0/pin/add`; my probe of `pin/add` stored an **empty file** and returned `QmdfTbBqBPQ7VNxZEYEj14VmRuZBkqFbiwReogJgS1zR1n` (the CID of `""`). Unknown routes such as `swarm/peers` return **HTTP 200 with an empty body** — any "is IPFS healthy?" check passes.
- No `/pin`, `/cat` URI-decoding issues aside, no auth, no pinning semantics, no peer identity derived from keys (peer IDs are hard-coded per port), binds `127.0.0.1` only.

`ipfs-hospitals/docker-compose.yml` — correct-looking 4-node Kubo cluster (5001-5004, gateways 8081-8084). Combined with the broken multipart upload in the pipeline, the "real" path has never plausibly been exercised.

`addToIPFS.py` — `docker cp` + `docker exec ipfs add`; works only on Linux/macOS hosts with those container names, no error aggregation, prints raw `ipfs add` output (which includes the file path, not just the CID).

3.5 Smart contracts

All four contracts compile cleanly with `solc 0.8.28` (optimizer 200 runs, **no errors, no warnings**) and the existing test suite passes: **14/14 in 231 ms**. Measured gas:

Function	Min	Max	Avg
`registerHospital`	79 132	96 244	84 840
`submitLocalModel`	177 817	232 481	~200 000
`publishGlobalModel`	88 272	129 226	~102 000
`ModelUpdateTracker` deployment	—	—	1 022 944

README claims "≈ 120 000 gas per submission" — the real figure is **~1.7-1.9×** that.

`ModelUpdateTracker.sol` (the only contract the docs treat as production) — findings verified by executing probe tests:

Probe	Result
Submit with **empty CID and zero hash**	**Succeeds**, counts toward quorum (`getRoundHospitals(1).length == 3`) — a hospital can satisfy the round with no artefact (`submitLocalModel` has no `require(bytes(cid).length > 0)`, unlike `ModelRegistry2`)
Publish global model with **empty CID**	**Succeeds** → round advances with an unusable pointer
`ModelUpdateTracker(0)`	**Deploys**; aggregator can `publishGlobalModel` with **0 submissions** and advance the round (`ModelRegistry2` does guard `requiredSubmissions > 0`; this one does not)
`setAggregator(address(0))`	**Accepted** → nobody can ever publish again; **no event**, no two-step handover
Change quorum later	**Impossible** — there is no `setRequiredSubmissions` (P3/P8 probes failed with "not a function")
Revoke a compromised hospital	**Impossible** — no `removeHospital`/`unregister`; 17 public functions, none can revoke
`pause()`	Owner can brick submissions indefinitely (no timelock, no multi-sig)
Round history	Per-round maps are keyed correctly, but a hospital's previous submission is only reachable via `getLocalModel(round, addr)`; there is no enumeration of submissions per round

`ModelRegistry2.sol` — richer (hospital allowlist, aggregator role, quorum, submitter list) but:

- constructor **auto-whitelists the deployer as a hospital** (verified `isHospital(owner) == true`);
- `publishGlobalModel(round, …)` can be called **repeatedly for the same round** and **silently overwrites** the recorded global CID (verified: `QmGLOBAL-1` → `QmGLOBAL-2-SILENTLY-REPLACED`). That destroys the "immutable audit trail" property the project is built on;
- `removeHospital` does **not** remove the hospital from `roundSubmitters`, so quorum accounting is wrong after a revocation (verified: still 3 after removal);
- no events for `setAggregator`; no `fileHash != 0` checks.

`ModelRegistry.sol` — owner-only, single submission per round (duplicate reverts with `Round already exists`, verified), **no way to correct/update a round**, no enumeration. Dead code relative to the pipeline.

`Lock.sol` — Hardhat boilerplate, unrelated to the project, still compiled and tested.

Centralization reality: `owner == aggregator == deployer` at deployment (verified by `deploy.js` output and the contract). Every "decentralized" guarantee reduces to trusting one EOA that can rotate the aggregator, pause the network, and publish arbitrary CIDs. There is no multi-sig, no governance, no timelock, no dispute window.

3.6 Dashboard backend (`server.py`)

141 lines, stdlib `HTTPServer` (**single-threaded**, contrary to README "multi-threaded"), binds `127.0.0.1:3000` (line 136). Live transcripts (Appendix A.6) confirm:

- `GET /api/status` returns a **module-level literal** (`fl_state`, line 38): round 2, accuracy 89.4, 186 000 samples, three named hospitals with specific CIDs, `loss: 0.042`. None of it is read from the chain, IPFS or the model.
- `POST /api/diagnose` does **token substring matching against disease names**, not symptoms: for every word >2 chars in the query it checks `word in disease_lower` and scores matches, then fabricates a confidence with `(score / total) * 94 + 2` (line 106). Real output:
  - input `"shortness of breath, chest tightness, wheezing, cough"` → `Open Wound Of The Chest` **49.0 %**, `Whooping Cough` **49.0 %**.
  - input `"heart failure"` → Heart Failure 33.3 %, Congenital Heart Defect 17.7 %, Heart Attack 17.7 % …
  - input `not json` (empty) → **a canned list**: Acute Bronchospasm 76.4 %, URI 18.2 %, Allergic Rhinitis 5.4 %.
- `POST /api/advance-round` increments a counter and adds **+1.8 % accuracy per click**, capped at 98.5. Unauthenticated. 22 curls moved the "system" from round 2 to round 24 at 98.5 % with zero training.
- No authentication, no CSRF/origin check, no CORS policy, no request-size limit (`Content-Length` is trusted), bare `except:` on JSON parsing, global mutable state mutated without a lock, no logging, no graceful shutdown, `import torch` and `import torch.nn` are **never used** (≈2 s startup + a 2 GB dependency for nothing).
- Path traversal is **blocked** (404 on `/../../etc/passwd`) — the one thing `SimpleHTTPRequestHandler` gives you for free.

3.7 Dashboard frontend

Design craft is genuinely good: light medical theme, 6 views, canvas charts, responsive layout, `styles.css` 2 350 lines. But it is a **static mockup wired to two DOM fields**:

- `updateBadges()` (app.js:40-57) updates only the version badge and one CID string. The hospital table, ledger, chart curves, participation bars, "14 hospitals", "1.2k updates", "124 hospitals", "1.2M gradients", "99.9 % Integrity" are **hard-coded in** `index.html`.
- The "Blockchain Transaction Ledger" shows fabricated transactions dated **2023-10-24** with placeholder hashes (`0xdeadbeef12345678990…`, `0xabcdef1234567890ab…`) and hospital names that do not exist in the API (`Mary_Center_01`, `Cleveland_C.Hospital_04`).
- "SHAP Value Consensus" is seven `div`s with hard-coded heights — no SHAP anywhere in the repo.
- Chart data are literal coordinate arrays (`points = [{x:0.05,y:0.15}, …]`).
- Failure mode: if `/api/status` is down, the UI silently keeps showing the fake numbers — the most dangerous possible behaviour for a clinical dashboard.
- Accessibility: **0** `aria-*` / `role=` attributes, emoji-as-icons, no `alt` text (no `<img>`), one external Google Fonts dependency (breaks offline/in sandboxed previews).
- XSS surface is small (disease names come from the local mapping via `innerHTML`), but `renderDiagnosis` interpolates server strings without escaping — one bad label edit becomes script execution.

3.8 Scripts & docs

- `deploy.js` deploys `ModelUpdateTracker(3)` and writes `deployment_info.json` — it does **not** register hospitals (README §10 Step 3 says it does).
- `add_hospitals.js` hard-codes `0x5FbDB…` (the `ModelUpdateTracker` address) but calls `getContractAt("ModelRegistry2", …)` → wrong ABI for that address.
- `interact.js` is the only end-to-end on-chain demo; it submits placeholder CIDs like `cid-Hospital1-r1` and `0x1111…` hashes.
- `test_ipfs_connection.js` has **unreachable code** (the retrieval block sits after a `return` in the fallback branch); `ipfs-http-client@60` is deprecated in favour of Helia.
- `deployment_info.json` is **committed** even though `Blockchain/.gitignore` ignores it; `.env.example` advertises `0xe7f1725E…` which matches **neither** the committed deployment nor the second Hardhat deploy address used in the docs. 13 places hard-code `0x5FbDB2315678afecb367f032d93F642f64180aa3`.
- `Blockchain/.gitignore` also ignores `IMPROVEMENTS_EXPLAINED.md`, `STEP_BY_STEP_GUIDE.md`, `IPFS_BLOCKCHAIN_INTEGRATION_GUIDE.md`, `requirements.txt` — yet all four are tracked. The ignore file was copied from another project.
- `FL/__pycache__/fl_server.cpython-310.pyc` is committed and references a module (`fl_server.py`, functions `load_vocab`, `train_client`, `fedavg`) that **no longer exists**.
- README is 902 lines of high-quality prose — roughly 30 % of it describes code that is not in the repository (Section 7).

---

4. Runtime verification (what was actually executed)

Check	Command	Result
Python syntax	`python3 -m compileall`	✔ clean
Solidity compile	`solc 0.8.28` (optimizer 200)	✔ 5 artifacts, **0 errors, 0 warnings**
Contract tests	`npx hardhat test --no-compile`	✔ **14 passing (231 ms)**
Security probes	custom `zz_probe.test.js` (deleted afterwards)	6 pass / 2 expected-fail → 8 confirmed defects (§3.5)
`ModelRegistry2` probes	custom test	✔ 4 confirmed defects (§3.5)
IPFS emulator	4 nodes on 5001-5004	✔ correct CIDv0 math · ✘ shared storage, `pin/add` hijacked by `/add`, unknown routes return 200
Dashboard API	`curl` against running server	✔ endpoints respond · ✘ data fabricated, diagnosis nonsense, round counter unbounded
Dashboard UI	served on `0.0.0.0:3000`	✔ renders (preview available)
Full pipeline	**not run**	requires `torch` (~2 GB) and is meaningless without real training data path

> Note: `npx hardhat test` cannot download the solc binary in a restricted network (`HH502`); the contracts were compiled with the npm `solc@0.8.28` package and the artifacts fed to Hardhat. This is an environment artefact, not a repo defect — but it also means **CI has never compiled these contracts**, since there is no CI.

---

5. Findings register

Severity: **C**ritical (invalidates the core claim / unsafe) · **H**igh · **M**edium · **L**ow.

Critical

ID	Finding	Evidence
C-01	**No federated learning actually happens.** Local "training" uses random tensors; the CSV is read and discarded; averaging three independently initialised random models yields a random model. No metric is computed.	`run_federated_pipeline.py:98,103-112,149-151`
C-02	**Fabricated metrics presented as measurements.** Dashboard shows 89.4 % accuracy; the only real number in the repo is 73.9 %. `/api/advance-round` adds +1.8 %/click up to 98.5 %.	`server.py:38,123-124`; notebook outputs; `README.md:323-341,527`
C-03	**Patient-shaped PII in Git.** Names, DOB, e-mail, phone, insurance, billing, emergency contacts committed as plain CSV, duplicated in `FL/` and `LocalTraining/`, permanent in history — in a project whose thesis is HIPAA/GDPR compliance.	`FL/Train&TestCSVs/*`, `LocalTraining/Train&TestCSVs/*`
C-04	`/api/diagnose` **produces clinically wrong output with confident percentages.** It substring-matches disease names, so dyspnoea returns "Open Wound Of The Chest 49 %". No model, no disclaimer, no uncertainty.	`server.py:89-114`; live transcript §A.6
C-05	**Smart contract cannot distinguish a real submission from an empty one.** Empty CID + zero hash is accepted, counts toward quorum, and an empty global CID can be published.	`ModelUpdateTracker.sol:108-133,135-160`; probe P1

High

ID	Finding	Evidence
H-01	**The pipeline never touches the blockchain.** No `web3`/`ethers` in Python; `deployment_info.json` is only printed; the "on-chain audit trail" is never written.	`run_federated_pipeline.py:66-70,88-89`; `grep web3` → 0 hits
H-02	**IPFS integration is broken against real Kubo** (raw body instead of multipart) and **silently degrades to fake, content-independent CIDs** that are not valid CIDs but are displayed as such.	`run_federated_pipeline.py:47-55,141-143`
H-03	`requiredSubmissions = 0` **is accepted**, allowing the aggregator to "complete" rounds with zero data.	`ModelUpdateTracker.sol:74-80`; probe P2
H-04	**Total admin centralization with no recovery path:** owner = aggregator = deployer; `setAggregator` accepts `address(0)` (permanent DoS) and emits no event; no hospital revocation; `pause()` unbounded; quorum immutable.	`ModelUpdateTracker.sol:74-80,95-101`; probes P4,P5,P7
H-05	**Unauthenticated state-changing API on a single-threaded server.** Anyone can advance rounds; no authz, CSRF, TLS, CORS, or input limits.	`server.py:11,118-128,136`
H-06	**Model artefacts are mutually incompatible** (`encoder.*` vs `feature_extractor.*`); the notebook weights can never be loaded by the pipeline, so the two halves of the "FL system" cannot be combined.	`LocalTrainingNotebook.ipynb` cell 5/9 vs `run_federated_pipeline.py:29-38`
H-07	`ModelRegistry2` **breaks immutability:** global CID for a round can be silently overwritten; removed hospitals remain counted in quorum.	`ModelRegistry2.sol:139-152`; probe
H-08	**Federated averaging degrades the model** (81.2 % local → 73.9 % global) and nothing in the design addresses it.	notebook outputs

Medium

ID	Finding	Evidence
M-01	Dashboard is a mockup: only 2 DOM fields are data-driven; hospitals, ledger, charts, SHAP, counts are literals (incl. 2023 timestamps and `0xdeadbeef…`).	`app.js:40-57`; `index.html:119-126,274,409-417,595-606,768-800`
M-02	Three competing registry contracts + boilerplate `Lock.sol`; `add_hospitals.js` uses the wrong ABI/address; `.env.example` disagrees with `deployment_info.json`.	`contracts/`, `scripts/add_hospitals.js:12-13`, `.env.example`
M-03	**No CI, no LICENSE, no Python/frontend tests**, `package.json test` script still the default `echo "Error: no test specified"`.	repo root
M-04	Repository hygiene: 67 MB of duplicated CSVs and notebooks; stale committed `.pyc`; docs tracked despite being gitignored; `deployment_info.json` both tracked and ignored; deploy screenshots committed.	`.gitignore`, `FL/__pycache__/`, `Blockchain/DeploymentPics/`
M-05	Reproducibility: no seeds, unpinned deps, Colab-only paths (`/content/drive/MyDrive/MedXChAIn`), Windows-flavoured emulator identity (`amd64/windows`), `e:/Fedacre/` in the README.	notebooks, `ipfs_service.py:70`, `README.md:137`
M-06	**Privacy guarantees are asserted, not implemented:** no DP-SGD/clipping/noise, no secure aggregation, no gradient encryption, no access control on IPFS; weights travel in clear text.	`README.md:341-348` vs codebase (0 hits for `noise`, `clip`, `opacus`)
M-07	Backend performance/robustness: single-threaded `HTTPServer`, unused `torch` import, unbounded `Content-Length`, bare `except`, global mutable state.	`server.py:11,79,83,136`
M-08	Data quality: 737/793 classes present, 68 singleton classes, long-tail → top-1 accuracy misleading; **7.2 %** exact symptom leakage train→test; IID splits only.	measured §3.3
M-09	No error handling in the pipeline; failure is the normal path; success banner is unconditional.	`run_federated_pipeline.py:136-146,192-196`
M-10	Documentation ↔ ABI mismatch: README documents a `ModelUpdate` struct, `submitUpdate(...)`, `RoundCompleted`, `getRoundSubmissions` — none exist in `ModelUpdateTracker.sol`.	`README.md:362-404,814-846`

Low

ID	Finding
L-01	Zero `aria-*`/`role=` attributes, emoji-only iconography, external Google Fonts (breaks offline).
L-02	Dead code: unreachable block in `test_ipfs_connection.js`, unused imports in `server.py` (`parse_qs`, `torch`, `nn`).
L-03	13 hard-coded contract addresses; no config module; no `.env` loading anywhere (`python-dotenv` absent).
L-04	No logging discipline (prints + emoji as the only observability); no structured audit log.
L-05	`ipfs-http-client` deprecated; `ipfs_service.py` emulator can mask genuine integration failures.
L-06	Notebooks shipped with outputs but stripped `execution_count`, and with a missing output for the main training cell — provenance is unverifiable.

---

6. Documentation drift matrix

README claim	Reality
"Only encrypted model parameter deltas leave the hospital"	Raw `.pt` weights; no encryption anywhere
"DP-SGD with gradient clipping + Gaussian noise"	Not implemented (no such code, no `opacus`)
"FedAvg weighted by sample count `n_k`"	Uniform `mean(dim=0)`; snippet in README does not match the file
"Pipeline dispatches on-chain transactions via JSON-RPC"	No blockchain client at all in Python
"deployment_info.json stores address **and full ABI**"	7-line JSON: address, owner, requiredSubmissions, network, timestamp
"deploy.js … registers the 3 hospital accounts"	It only deploys
"Contract ≈120 000 gas per submission"	177 817–232 481 (avg ≈200 k)
"ModelUpdateTracker has `submitUpdate`, `RoundCompleted`, `getRoundSubmissions`"	Those symbols do not exist
"ModelRegistry2 provides penalty scoring and Byzantine fault tolerance"	No penalty, no BFT, no reputation
"Multi-threaded HTTP server"	`HTTPServer`, single-threaded
"793-class ontology: index 0 = Cardiac Arrhythmia, 14 = Hypertension, 42 = Acute Bronchospasm, 89 = T2DM, 105 = Sleep Apnea, 240 = Migraine"	Actual: 0 = abdominal aortic aneurysm, 14 = acute glaucoma, 42 = anemia due to malignancy, 89 = brain cancer, 105 = cataract, 240 = esophageal cancer. (Cardiac arrhythmia = 52; migraine = 465; hypertension = 360/361/362/424/453)
"Model: 7.2 M parameters"	`MedCPTClassifier` = **800 537** params; MedCPT encoder ≈110 M
"SHAP value consensus"	No SHAP; bars are hard-coded `div` heights
"train_1..3 hold 62 000 records each, N = 186 000"	49 613 train + 12 387 test = 62 000 per hospital (186 000 incl. test); pipeline reads 500 rows and ignores them
"89.4 % converged accuracy"	Only measured value: 73.8–74.0 % (global), 81.23 % (single local)
"IPFS CIDs verified: Yes"	Printed unconditionally; with Docker absent the CIDs are fabricated strings
"Dashboard live-syncs blockchain ledger"	Static HTML table dated 2023-10-24

---

7. Privacy, safety and legal

1. **PHI-shaped data in a public repo.** Even if synthetic, the file format trains contributors to handle real PHI carelessly; if any row is real, this is a HIPAA/GDPR incident. Recommendation: keep only schema + generator upstream, distribute data out-of-band (encrypted bucket / LFS with access control), and scrub history (`git filter-repo`) before any public release.
2. **No privacy mechanism despite the title.** Federated learning alone is not privacy: raw weights are invertible (deep leakage from gradients), and here weights are additionally published to IPFS with no access control. Minimum viable: DP-SGD (per-example clipping + calibrated noise, `opacus`), secure aggregation, and encrypted IPFS payloads (e.g. `age`/AES-GCM with per-round keys).
3. **Clinical decision support without a disclaimer.** The diagnosis endpoint returns "confidence" percentages from a substring match. Any UI screenshot of it is a liability. Add a persistent "research prototype — not for clinical use" banner and gate the endpoint behind a demo flag.
4. **Audit-trail integrity is weaker than advertised.** Between empty CIDs (C-05) and silent overwrite (H-07), an on-chain "record" can be produced for an artefact that does not exist, or replaced after the fact.
5. **License ambiguity.** README badge says MIT, repository has no `LICENSE`. Add one, and a `DATA_LICENSE`/provenance note for the CSVs.

---

8. Performance & scalability observations

- **Per-round traffic:** realistic deployment would move the full MedCPT encoder (≈110 M params ≈ 440 MB fp32) per hospital per round — 1.3 GB/round for 3 hospitals, over IPFS HTTP, at 10 s timeouts. The pipeline's 0.8 M-param toy (3.2 MB) hides this entirely. Real systems aggregate adapters/LoRA or use compression + quantization.
- **Gas:** ~200 k per submission × 3 hospitals + ~100 k publish ≈ 0.7 M gas/round. Fine on a local chain, ~0.02 ETH/round on mainnet at 20 gwei — economical only because nothing is actually stored (CID strings + hash).
- **Dashboard:** single-threaded server; the `torch` import adds seconds of startup and a 2 GB dependency for zero functionality; charts recompute on every tab switch.
- **Data loading:** `read_csv` without `usecols`/`dtype` on 18 MB × 31-column files; the pipeline loads 4.6-18 MB files to use none of the content.
- **Chain writes are not batched** (no Merkle root per round) — a Merkle-root-per-round design would cut cost by ~10× and strengthen the audit trail.

---

9. What is genuinely good

- **Contract engineering basics are sound:** clean compile with zero warnings, sensible modifiers, per-round replay protection (`hasSubmitted`), role separation in `ModelRegistry2`, real unit tests for the lifecycle, correct use of `bytes32` for hashes.
- **CIDv0 implementation is correct** — the base58/multihash math is right (verified against an independent computation).
- **Dashboard craft:** coherent design system, 6 well-structured views, responsive layout, no framework bloat.
- **Documentation ambition:** the README explains the architecture, the math and the runbook better than most student projects; it is the claims, not the writing, that need fixing.
- **Clear module boundaries** (`FL/`, `Blockchain/`, `ipfs-hospitals/`, `UI-DashBoard/`) make the project easy to fix incrementally.

---

10. Prioritized remediation plan

P0 — make it honest and safe (1-2 days)

1. **Stop lying in the UI and README.** Gate fabricated metrics behind a `DEMO_MODE` flag; replace the 89.4 % with either a measured value or "—". Add a "research prototype, not for clinical use" banner on the diagnosis view.
2. **Remove the diagnosis endpoint or make it real.** Until a real model is wired, return HTTP 501 or run the actual classifier. Never return substring-match percentages.
3. **Strip PII columns from committed data** (`Patient_*`, `Doctor_*`, `Billing`, emergency contacts) and move datasets out of Git (LFS/bucket); add `git filter-repo` scrub before publishing.
4. **Harden the contract** (`ModelUpdateTracker.sol`):

``solidity constructor(uint256 _requiredSubmissions) { require(_requiredSubmissions > 0, "quorum=0"); ... } function submitLocalModel(string calldata cid, bytes32 fileHash) external ... { require(bytes(cid).length > 0, "empty cid"); require(fileHash != bytes32(0), "empty hash"); ... } function setAggregator(address a) external onlyOwner { require(a != address(0), "zero addr"); emit AggregatorChanged(aggregator, a); aggregator = a; } function setRequiredSubmissions(uint256 n) external onlyOwner { require(n > 0); ... emit } function removeHospital(address h) external onlyOwner { ... emit } function publishGlobalModel(...) external onlyAggregator { require(bytes(globalModels[currentRound].cid).length == 0, "already published"); ... } ` Add Ownable2Step, a whenNotPaused guard on registerHospital`, and tests for every new revert.

5. **Fix the IPFS client:** multipart upload with `pin=true`, verify the returned CID against the local SHA-256, raise on mismatch, and **never invent a CID** — surface the failure instead.

P1 — close the loop (1-2 weeks)

6. **Actually submit to the chain from Python** (`web3.py`): register → `submitLocalModel(cid, sha256)` → wait for receipts → aggregator `publishGlobalModel`; store tx hashes in a round manifest. Delete the hard-coded address; read it from `deployment_info.json`/env.
7. **Reconnect the dashboard to reality:** read the contract (`currentRound`, `getRoundModel`, events) and IPFS CIDs server-side, expose them via `/api/status`, and render hospital/ledger tables from JSON instead of literals. If a dependency is down, show an error state — never cached fake data.
8. **Rebuild the training path:** real `symptoms → label` training (the notebook code is the starting point), one checkpoint format shared by pipeline and notebooks (`encoder + head`), fixed seeds, and a saved `metrics.json` per round (macro-F1, top-5, per-class recall).
9. **Server hygiene:** `ThreadingHTTPServer`, auth token for POSTs, `Content-Length` cap, JSON validation, structured logging, remove the unused `torch` import, bind configurable host/port.
10. **Repo hygiene:** delete `LocalTraining/` duplicate tree (or symlink), drop `FL/__pycache__`, remove `Lock.sol`/`Lock.js`, unify the three registry contracts into one, add `LICENSE`, fix both `.gitignore` files, remove the 2023/fake ledger rows.

P2 — make it credible (2-8 weeks)

11. **Differential privacy** (Opacus DP-SGD with a stated ε,δ) or **remove the DP claims**; add secure aggregation and encrypted IPFS blobs.
12. **Address non-IID:** Dirichlet label-skew partitions, FedProx/SCAFFOLD baselines, and an honest local-vs-global accuracy table — the current 81.2 % → 73.9 % regression must be explained or fixed.
13. **Byzantine robustness:** reputation/stake, hash-verified model fetching, and a "reject update" path (currently impossible on-chain).
14. **CI:** GitHub Actions with `hardhat test`, `solc` compile, `ruff`, `mypy`, `pytest` (API tests with `pytest-httpx`), `solhint`, and a dependency audit.
15. **Observability & ops:** Prometheus metrics, round manifests, a Dockerfile for the app + compose profile for the full stack, and a single `make dev` entrypoint instead of six manual terminals.

Appendix A — Reproducible command log

A.1 Compilation (network-restricted workaround)

```
npm install --no-save solc@0.8.28
# custom script compiled contracts/ with optimizer(200) and emitted
# Hardhat-format artifacts to Blockchain/artifacts/contracts/**.json
# → 5 artifacts, 0 errors, 0 warnings
```

A.2 Test suite

```
npx hardhat test --no-compile     # → 14 passing (231ms)
```

A.3 Contract probes (temporary, deleted after running)

```
P1 empty CID accepted; quorum = 3; global published with empty cid, round → 2
P2 ModelUpdateTracker(0) deploys; publish with 0 submissions, round → 2
P3 setRequiredSubmissions does not exist (TypeError)
P4 aggregator → 0x000…000 (no event)
P5 functions: … no removeHospital / unregister
P6 per-round overwrite works; totalSubmissions increments to 2
P7 pause blocks submissions
ModelRegistry2: deployer auto-whitelisted = true
ModelRegistry2: round1 cid QmGLOBAL-1 → QmGLOBAL-2-SILENTLY-REPLACED
ModelRegistry2: submitters after removal = 3
ModelRegistry: blocks re-submission same round = true
```

A.4 IPFS emulator

```
expected cid: QmYjyGJrNaqN9RuKUjW3QKtRZNNe2n2aiHTan9r3Lx1R2L (len 46)
add → same hash (✔ math correct)
cat from 5004 of file added on 5001 → b'hello model weights'  (✘ shared storage)
POST /api/v0/pin/add?arg=…  → stored an empty file, returned QmdfTbBq… (✘ /add suffix match)
POST /api/v0/swarm/peers    → HTTP 200, empty body (✘)
```

A.5 Gas report (from `hardhat test`)

```
registerHospital   79 132 — 96 244   avg 84 840
submitLocalModel  177 817 — 232 481
publishGlobalModel 88 272 — 129 226
ModelUpdateTracker deployment ≈ 1 022 944 (3.4 % of block gas limit)
```

A.6 Dashboard API transcript

```
GET /api/status → {"currentRound":2,"globalAccuracy":89.4,...,"loss":0.042,...}   (static literal)

POST /api/diagnose {"symptoms":"shortness of breath, chest tightness, wheezing, cough"}
 → {"results":[{"disease":"Open Wound Of The Chest","confidence":49.0},
               {"disease":"Whooping Cough","confidence":49.0}]}

POST /api/diagnose {"symptoms":"heart failure"}
 → Heart Failure 33.3 | Congenital Heart Defect 17.7 | Heart Attack 17.7 | Heart Block 17.7 | Heart Contusion 17.7

POST /api/diagnose (body "not json")
 → Acute Bronchospasm 76.4 | Upper Respiratory Tract Infection 18.2 | Seasonal Allergic Rhinitis 5.4

POST /api/advance-round ×22 → round 24, globalAccuracy 98.5  (no auth, no training)
GET /../../../../etc/passwd → 404 (path traversal correctly blocked)
GET /api/labels → 22 925 bytes
GET /           → 200
```

A.7 Dataset measurements

```
train_1 49 613 rows | train_2 49 614 | train_3 49 613
test_1  12 387 rows | test_2  12 386 | test_3  12 387
columns 31 · labels 0-791 · 737 classes in train_1 · 68 singleton classes · max class 350
hospital partitions mutually disjoint (0 exact-row overlap)
test_1 symptom strings duplicated from train_1: 889 / 12 387 (7.2 %)
```

A.8 Notebook outputs (as shipped)

```
LocalTraining … cell 9:  Test Accuracy: 81.23 %
FL/Global… cell 4:  Hospital 1 → 73.84 % | Hospital 2 → 73.87 % | Hospital 3 → 73.97 %
(cell 8, the 10-epoch training loop, has no stored output)
```

Appendix B — Artefacts produced with this audit

File	Purpose
`TECHNICAL_AUDIT.md`	This document — evidence, findings register, claim-vs-reality matrix
`ENHANCEMENT_ROADMAP.md`	What to build: 12 quick wins, 11 tracks, 18 new features, SLOs, first 10 PRs
`AGENT_BRIEF.md`	Working contract for the implementing agent: rules, gates, landmines, definition of done
`agent_backlog.yaml`	Machine-readable backlog: 18 tasks with acceptance criteria, verify commands, stop conditions
`MEDXCHAIN_AUDIT_AND_ROADMAP.pdf`	The three documents above as one navigable PDF (regenerate with `tools/md_to_pdf.py`)
`tools/md_to_pdf.py`	Reproducible Markdown → PDF builder (reportlab)

Temporary probe tests (`Blockchain/test/zz_probe*.test.js`) and the `solc` compile helper used during the audit were removed; `Blockchain/node_modules/` and `Blockchain/artifacts/` are gitignored build output.

Part II — Enhancement & New-Feature Plan

What to build, in what order, and exactly how — every item sized, sequenced and tied to a finding from Part I. **Companion to `TECHNICAL_AUDIT.md`.** The audit said what is broken; this document says what to build, in what order, and exactly how.

0. How to read this

Three reading paths:

You have…	Read	Outcome
1–2 days	§1 (quick wins) + §17 (first 10 PRs)	Project stops lying and becomes defensible in a demo
2–4 weeks	Tracks A, C, E (core loop)	System actually trains, actually talks to the chain, dashboard shows real data
One semester / 12 weeks	Everything	Publication-grade research prototype + credible product

Effort is in **person-days (pd)** for a competent developer who knows the codebase. Priority: **P0** = correctness/safety, **P1** = closes the loop, **P2** = product credibility, **P3** = research/stretch.

1. The 12 quick wins (start today, ~8 pd total)

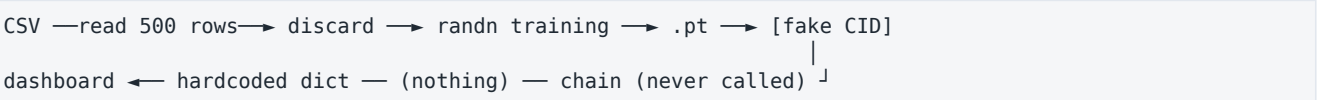
These are cheap, high-signal, and unblock everything else.

#	Change	File	pd	Closes
1	Delete fabricated metrics from `fl_state`; return `null` + `"source": "unavailable"` when a dependency is down	`server.py:38`	0.5	C-02
2	Add `require(bytes(cid).length > 0)` + `fileHash != 0` + `_requiredSubmissions > 0`	`ModelUpdateTracker.sol`	0.5	C-05, H-03
3	Replace substring diagnosis with **HTTP 501 "model not wired"** until Track G lands	`server.py:89-114`	0.5	C-04
4	Remove PII columns from the committed CSVs (keep `symptoms,label,Disease_name` only)	`FL/Train&TestCSVs/*`	1	C-03
5	Delete the duplicated `LocalTraining/Train&TestCSVs/` tree (67 MB → 33 MB)	—	0.2	M-04
6	Remove `FL/__pycache__/` and add `fl_server` note	—	0.1	M-04
7	Fix `upload_to_ipfs` to multipart + verify CID, **never invent a CID**	`run_federated_pipeline.py:47-55`	1	H-02
8	Seed everything (`random`, `numpy`, `torch`), log the seed in the round manifest	pipeline + notebooks	0.5	M-05
9	Add `LICENSE` (MIT) and a `DATA_PROVENANCE.md`	root	0.2	M-03
10	Add "Research prototype — not for clinical use" banner to the diagnosis view	`index.html`	0.3	§3.7
11	Fix `.gitignore` (un-ignore the docs it currently hides), stop tracking `deployment_info.json`	`Blockchain/.gitignore`	0.2	M-04
12	Add GitHub Actions: `ruff`, `pytest`, `hardhat test`, `slither`	`.github/workflows/ci.yml`	1.5	M-03

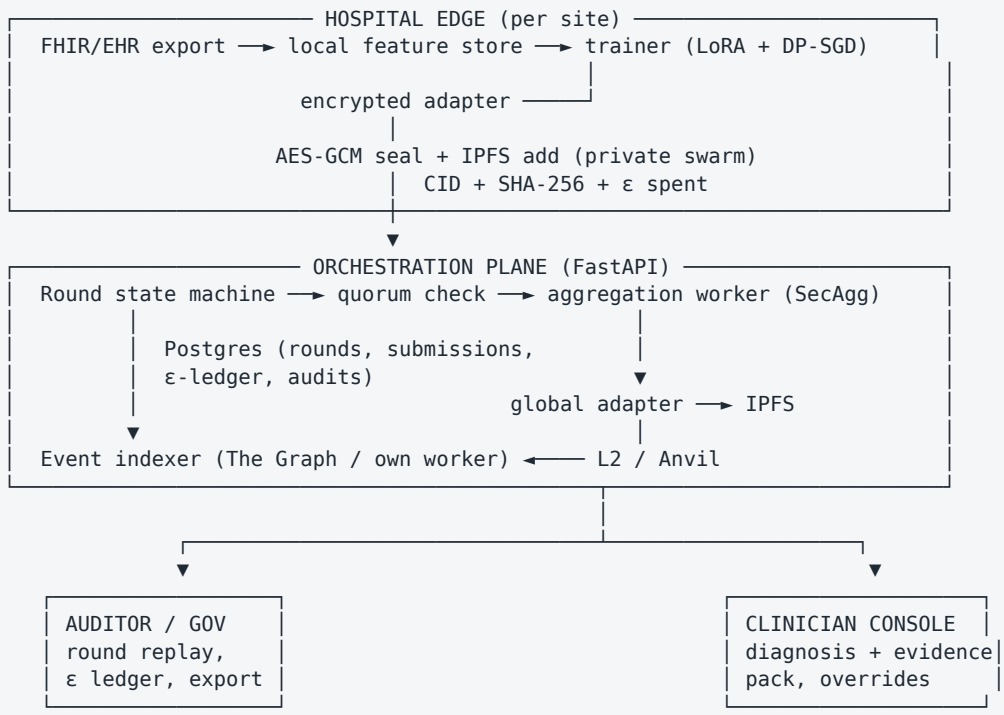
Everything below builds on these.

2. North-star architecture

Today



Target



Key architectural shifts:

1. **One source of truth = Postgres.** JSON files (`deployment_info.json`, `fl_state`) become outputs, never state.
2. **Chain = audit anchor, not database.** One Merkle root per round + settlement events, not per-tensor writes.
3. **IPFS = transport of ciphertext.** Nothing readable is ever pushed.
4. **Dashboard = a client.** Zero literals; every number traceable to a query, with `provenance` metadata.
5. **Adapters, not full models.** LoRA makes communication, encryption and DP practically feasible (0.3 MB vs 440 MB).

---

3. Target repository layout

```
medxchain/
├── pyproject.toml                    # single source of deps, ruff/mypy/pytest config
├── Makefile                          # make dev | test | lint | up | seed | round
├── docker-compose.yml                # anvil + 4 kubo + cluster + pg + redis + api + worker + ui + obs
├── .env.example
├── LICENSE
├── README.md                         # rewritten, claim-verified
├── docs/
│   ├── architecture.md  threat-model.md  privacy.md  compliance.md  runbook.md  adr/
├── src/medxchain/
│   ├── config.py                     # pydantic-settings
│   ├── logging.py                    # structlog, JSON, trace ids
│   ├── db/           models.py  session.py  migrations/
│   ├── chain/        client.py  abi/  events.py  indexer.py
│   ├── storage/      ipfs.py  crypto.py  verify.py
│   ├── fl/           model.py  lora.py  data.py  trainer.py  aggregator.py  dp.py  secagg.py  metrics.py
│   ├── rounds/       state.py  service.py  workers.py  manifest.py
│   ├── api/          app.py  deps.py  routes/{status,hospitals,rounds,diagnose,audit,admin}.py
│   ├── clinical/     infer.py  calibrate.py  conformal.py  explain.py  icd10.py  guardrails.py
│   └── cli.py                        # medxchain rounds start|seal|finalize, doctor check
├── contracts/                        # Foundry (replaces Hardhat)
│   ├── FederatedRound.sol
│   ├── test/FederatedRound.t.sol     # fuzz + invariant tests
│   └── script/Deploy.s.sol
├── tests/                            # pytest: unit, property, integration, e2e
├── web/                              # Vite + vanilla/React, no fabricated data
└── notebooks/                        # moved out of the runtime path, outputs cleared
```

Why: one package namespace kills the `sys.path` hacks, makes imports testable, and lets `pip install -e .` replace "somehow run this from the repo root".

---

4. Phase plan at a glance

Phase	Weeks	Tracks	Exit criteria
0 — Honesty	1	§1 quick wins	No fabricated number anywhere; CI green; PII gone from Git
1 — Loop closes	2–4	A (core), C (v2 contract), E (service+DB)	One command runs a round end-to-end; every dashboard number comes from Postgres
2 — Privacy	5–6	B (DP-SGD, SecAgg, encrypted blobs)	ε reported per round; ciphertext-only IPFS; MIA AUC measured
3 — Clinical	7–8	G (calibration, conformal, abstention, ICD-10, explain)	Diagnosis output clinically sane + explains itself + refuses when unsure
4 — Product	9–10	F (dashboard 2.0), H (features), J (compliance pack)	Auditor can replay any round; ε ledger visible per site; signed audit PDF exported
5 — Rigor	11–12	L (benchmarks), K (ops), I (security)	FedAvg/FedProx/SCAFFOLD table on non-IID splits; threat model signed off

Parallelizable by two people: **Dev 1 = A + G + L**, **Dev 2 = C + E + F**; B and I touch both.

---

5. Track A — Make the federated learning real

A1. Data layer (2 pd) — closes C-01, M-08

Problems to fix: pipeline reads 500 rows and ignores them; IID splits; 7.2 % test leakage; PII columns; long-tail classes.

Do this:

```
# src/medxchain/fl/data.py
REQUIRED = {"symptoms", "label"}
FORBIDDEN = {"Patient_First_Name", "Patient_Last_Name", "Patient_Email",
             "Patient_Phone_Number", "Patient_Date_Of_Birth", "Patient_Insurance_Provider",
             "Patient_Emergency_ContactEmail", "Patient_Emergency_Contact_PhoneNumber"}

def load_site(path: Path) -> pd.DataFrame:
    df = pd.read_csv(path, usecols=["symptoms", "label"], dtype={"symptoms": "string", "label": "int32"})
    assert REQUIRED <= set(df.columns), f"missing columns in {path}"
    assert not (FORBIDDEN & set(df.columns)), "PII columns must be stripped before packaging"
    df = df.dropna(subset=["symptoms"]).drop_duplicates(subset=["symptoms", "label"])
    return df

def split_site(df, seed=42, test_size=0.2):
    return train_test_split(df, test_size=test_size, random_state=seed, stratify=None)  # stratify fails on singletons
```

Then add:

- **Non-IID partitioner** (the single most important scientific upgrade):

```
def dirichlet_partition(df, n_sites=3, alpha=0.3, seed=42):
    """Label-skew partition. alpha→0 = each site owns few classes (pathological)."""
    rng = np.random.default_rng(seed)
    parts = [[] for _ in range(n_sites)]
    for label, grp in df.groupby("label"):
        p = rng.dirichlet([alpha] * n_sites)
        idx = rng.permutation(len(grp))
        cuts = (np.cumsum(p) * len(grp)).astype(int)[:-1]
        for site, chunk in enumerate(np.split(idx, cuts)):
            parts[site].append(grp.iloc[chunk])
    return [pd.concat(p).sample(frac=1, random_state=seed) for p in parts]
```

- **Dedup across splits** (kills the 7.2 % leak): hash `symptoms`, drop test rows whose hash exists in any train split.
- **Class-imbalance handling:** report macro-F1; merge the 68 singleton classes into an `other` bucket or exclude them from top-1 and report separately. Do not silently keep a 793-way problem where 68 classes have one example.
- **Dataset card** (`docs/dataset.md`): provenance, licence, row counts, label histogram, known biases, PII statement.

Acceptance: `medxchain data validate` fails the build if PII columns or cross-split duplicates exist; a stored `data_stats.json` lists per-site row counts and label coverage.

A2. Model & checkpoint contract (2 pd) — closes H-06

Today the notebook model (`encoder.*`, `classifier.0.*`, `classifier.3.*`) and pipeline model (`feature_extractor.*`, `classifier.*`) are load-incompatible. Define **one** artefact format:

```
# src/medxchain/fl/model.py
@dataclass
class Checkpoint:
    format: str = "medxchain-ckpt/1"
    base_model: str = "ncbi/MedCPT-Query-Encoder"
    num_labels: int = 793
    tunable: str = "lora"            # "lora" | "head" | "full"
    seed: int = 0
    round: int = 0
    site_id: str = ""
    state_dict: dict[str, Tensor] = field(default_factory=dict)
    metrics: dict[str, float] = field(default_factory=dict)

    def save(self, path: Path) -> str:      # returns sha256 hex
        torch.save(asdict(self), path)
        return sha256_file(path)
```

Rules: every artefact carries `format`, `base_model`, `num_labels`, `tunable`, `seed`, `site_id`, `round`, `metrics`. `load_checkpoint()` **rejects** mismatched `base_model`/`num_labels`/`tunable` with a clear error. That single guard would have prevented the entire notebook/pipeline divergence.

Add LoRA (this is the highest-leverage ML change in the whole plan):

```
from peft import LoraConfig, get_peft_model

def build_model(cfg) -> nn.Module:
    encoder = AutoModel.from_pretrained(cfg.base_model)
    if cfg.tunable == "lora":
        encoder = get_peft_model(encoder, LoraConfig(
            r=8, lora_alpha=16, lora_dropout=0.05,
            target_modules=["query", "value"], bias="none",
            task_type="FEATURE_EXTRACTION"))
    return MedCPTClassifier(encoder, cfg.num_labels, freeze_encoder=(cfg.tunable != "full"))

def trainable_state(model) -> dict[str, Tensor]:
    return {k: v.detach().clone() for k, v in model.state_dict().items() if is_trainable(model, k)}
```

Numbers: full MedCPT encoder ≈ 110 M params ≈ **440 MB** fp32 per hospital per round (1.3 GB/round for 3 sites). LoRA r=8 on q,v of 12 layers ≈ **295 k params ≈ 1.2 MB fp32 / 0.3 MB int8** — a **~370×** reduction. Everything downstream (IPFS latency, encryption cost, DP noise scale, HE feasibility, gas, dashboard transfer) gets 2–3 orders of magnitude easier. Do this before anything else in Track A.

Acceptance: `tests/test_checkpoint.py` round-trips, rejects mismatched format, and asserts `param_count(lora) / param_count(full) < 0.01`.

A3. Real local training (3 pd) — closes C-01, M-05, M-09

Replace the `randn` loop with an actual loop, using the same function in the pipeline and in the hospital SDK:

```
# src/medxchain/fl/trainer.py
def local_train(model, loader, *, epochs, lr, device, dp=None, global_state=None, mu=0.0, seed=0) -> tuple[nn.Module, dict]:
    set_seed(seed)
    model.to(device).train()
    opt = torch.optim.AdamW([p for p in model.parameters() if p.requires_grad], lr=lr)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
    if dp:                                          # Track B
        model, opt, loader, dp_ctx = dp.wrap(model, opt, loader, epochs)
    crit = nn.CrossEntropyLoss()
    for ep in range(epochs):
        for batch in loader:
            ids, mask, y = (t.to(device) for t in (batch["input_ids"], batch["attention_mask"], batch["label"]))
            logits = model(ids, mask)
            loss = crit(logits, y)
            if mu and global_state is not None:    # FedProx proximal term
                loss = loss + (mu / 2) * sum(((p - global_state[k].to(device)) ** 2).sum()
                                             for k, p in trainable_named(model))
            opt.zero_grad(set_to_none=True); loss.backward()
            torch.nn.utils.clip_grad_norm_([p for p in model.parameters() if p.requires_grad], 1.0)
            opt.step()
        sched.step()
        log.info("local_epoch", epoch=ep, loss=float(loss))
    metrics = evaluate_local(model, loader, device)   # A5
    metrics["epsilon"] = dp_ctx.epsilon if dp else 0.0
    metrics["num_samples"] = len(loader.dataset)
    return model, metrics
```

Hard requirements:

- `num_samples` is recorded and used for weighted aggregation (fixes the README/implementation mismatch).
- Every hyper-parameter comes from config, not a literal inside the function.
- The function returns metrics; it never prints and pretends.
- Exceptions propagate with context (site id, round, stage) — remove the `try/except: use fake CID` pattern.

Acceptance: with a fixed seed, two runs produce byte-identical checkpoints (test); `metrics.json` exists per site per round; the pipeline fails loudly if a site cannot train.

A4. Aggregation service (3 pd) — closes M-01 (weighting), H-08

```
# src/medxchain/fl/aggregator.py
def fedavg(updates: list[Checkpoint], *, weights: list[float] | None = None) -> dict:
    assert len(updates) > 0
    w = np.array(weights if weights is not None else [u.metrics["num_samples"] for u in updates], dtype=np.float64)
    assert w.sum() > 0
    w = w / w.sum()
    keys = set(updates[0].state_dict)
    assert all(set(u.state_dict) == keys for u in updates), "incompatible updates"
    return {k: sum(torch.as_tensor(u.state_dict[k]).float() * w[i] for i, u in enumerate(updates)).to(updates[0].state_dict[k].dtype)
            for k in keys}
```

Add, behind flags (each is ~0.5 pd and gives you an ablation table for Track L):

- `weighting: uniform | samples | quality` — quality uses `n_k / loss_k` clamped, documented as a heuristic.
- `fedprox` — `mu` penalty in the client (above).
- `scaffold` — control variates (2–3 pd, only if you want a stronger paper).
- **Robust aggregation** — `trimmed_mean`, `coordinate_median`, `krum`. Required for any Byzantine claim:

```
def trimmed_mean(updates, trim=0.2):
    stacked = torch.stack([flatten(u.state_dict) for u in updates])
    k = max(0, int(len(updates) * trim))
    values = stacked.sort(dim=0).values
    return unflatten(values[k:len(updates)-k].mean(0))
```

- **Straggler policy** — `ceil(2K/3)` quorum with timeout, record which sites missed the round (`missing: [3]`) so the audit trail is honest about partial participation.

Acceptance: property test — `fedavg` is invariant to update order, weights sum to 1, and `fedavg([u,u,u]) == u`; a 3-site round with one dropout still finalises and records `missing`.

A5. Evaluation harness (2 pd) — closes C-02, M-08

Stop reporting one number. Produce per round:

```
{
  "round": 4, "model": "v4.0.4", "timestamp": "...", "n_train": 148839,
  "global": {"accuracy": 0.74, "macro_f1": 0.51, "top5_accuracy": 0.88,
             "ece": 0.071, "coverage@0.9": 0.905, "set_size@0.9": 3.4},
  "per_site": {"h1": {...}, "h2": {...}, "h3": {...}},
  "worst_site": "h3", "worst_class_f1": 0.0, "classes_with_support<5": 68,
  "baselines": {"local_only_h1": 0.812, "centralized": null},
  "delta_vs_previous": {"macro_f1": +0.02}
}
```

Compute: accuracy, **macro-F1** (the honest metric for 793 imbalanced classes), top-5, per-class recall, **ECE** (calibration), **conformal coverage/set size**, confusion-matrix hash for drift detection.

Also store the **81.23 % local vs 73.9 % global** comparison as a first-class baseline and display it. Federation currently hurts — hiding that is the biggest scientific credibility problem in the repo.

Acceptance: `/api/rounds/{id}/metrics` returns this object; the dashboard charts it; a regression beyond `−0.02` macro-F1 fails CI on the golden dataset.

A6. Round manifest (1 pd) — new

Every round writes an immutable bundle:

```
artifacts/rounds/0004/
├── manifest.json      # inputs: site ids, seeds, hyper-params, code git sha, container digests
├── metrics.json
├── submissions.json   # site → {cid, sha256, num_samples, epsilon, tx_hash}
├── global.json        # cid, sha256, tx_hash, merkle_root
└── env.lock           # pip freeze + npm ls
```

`manifest.json` must include `git_sha`, `image_digest`, and `seed` so a third party can reproduce. This is what turns "demo" into "research artefact".

---

6. Track B — Privacy engineering (the title finally becomes true)

Today: no DP, no secure aggregation, no encryption, no access control. Three claims in the README are simply false.

B1. DP-SGD (3 pd) — closes M-06

```
# src/medxchain/fl/dp.py
from opacus import PrivacyEngine

@dataclass
class DPContext:
    epsilon: float; delta: float; max_grad_norm: float

def wrap(model, opt, loader, epochs, *, target_eps, delta, clip):
    engine = PrivacyEngine(secure_mode=False)
    model, opt, loader = engine.make_private_with_epsilon(
        module=model, optimizer=opt, data_loader=loader, epochs=epochs,
        target_epsilon=target_eps, target_delta=delta,
        max_grad_norm=clip, grad_sample_mode="ghost")  # ghost = fast clipping
    return model, opt, loader, DPContext(target_eps, delta, clip)
```

Practical notes that will save you a week:

- **DP over LoRA adapters only** (A2) is the only way this runs in reasonable time — ghost clipping over 110 M params on CPU is hopeless.
- Batch size must be ≥ ~1 000 for meaningful ε at 5 epochs. Report the **ε actually spent**, per site, per round, in the manifest.
- `grad_sample_mode="ghost"` requires a recent Opacus; pin it.
- **Never** write "HIPAA compliant" because of DP. Say "ε = 7.8 (δ = 1e-5) under DP-SGD, which bounds membership-inference advantage" — a checkable statement.

Acceptance: `metrics.json` contains `epsilon`, `delta`, `noise_multiplier`; a test asserts ε grows monotonically across rounds; the ε ledger (feature F2) reads from here.

B2. Secure aggregation (4 pd)

Server must never see an individual adapter. Simplest correct design: **pairwise additive masking** (SecAgg-lite).

```
def mask_update(update: dict[str, Tensor], my_id, peer_ids, shared_secrets, round_id, shape) -> dict:
    """Each pair derives a deterministic PRG stream; masks cancel when summed."""
    masked = {k: v.clone() for k, v in update.items()}
    for peer in peer_ids:
        if peer == my_id: continue
        stream = prg(shared_secrets[peer], round_id, shape)   # SHAKE256 → int8 vector
        sign = +1 if my_id < peer else -1
        for k in masked: masked[k] += sign * stream[k]
    return masked
```

Server sums the masked tensors; pairwise masks cancel exactly (up to float drift — use int64 quantised fixed-point, which also gives you **quantisation for free**). Add dropout handling: a dropped client's mask must be cancelled by the surviving peers, otherwise the sum is wrong — implement the standard `recover` step, or restrict to synchronous rounds with a timeout and exclude non-responders before aggregation (much easier, acceptable for 3–5 sites).

Alternatives and when to prefer them:

Option	When	Cost
Pairwise masking (above)	Default; 3–10 sites, synchronous	4 pd, exact
TenSEAL CKKS HE	You need aggregation over encrypted values and want a paper	8 pd; feasible **only** with LoRA-sized tensors
TEE (SGX/SEV)	You control the aggregator host	5 pd + infra; trust assumption changes
Trusted aggregator + DP only	Demo, deadline	0 pd, weakest guarantee — say so

Acceptance: an integration test proves the server-visible tensor sum equals the plaintext sum within tolerance, and that no single masked update can be inverted.

B3. Encrypt everything before IPFS (2 pd) — closes H-02/M-06

```
# src/medxchain/storage/crypto.py
def seal(plaintext: bytes, key: bytes, aad: bytes = b"medxchain/v1") -> bytes:
    nonce = os.urandom(12)
    return nonce + AESGCM(key).encrypt(nonce, plaintext, aad)

def open_(blob: bytes, key: bytes, aad: bytes = b"medxchain/v1") -> bytes:
    return AESGCM(key).decrypt(blob[:12], blob[12:], aad)
```

Key management options, in order of practicality:

1. **Per-round data key**, wrapped for each participant with X25519 (`cryptography` lib) — recipients unwrap, IPFS holds only ciphertext.
2. **External KMS / Vault** for the demo; document the trust boundary.
3. **Crypto-shredding for GDPR**: keep the wrapping key off-chain in your DB; deleting the key makes the on-chain hash useless → satisfies erasure without mutating the chain. **This is the cleanest answer to "right to erasure vs immutable ledger"** and belongs in `docs/compliance.md`.

Also: hash the ciphertext, publish `sha256(ciphertext)` on-chain, verify after download before decrypting. Store `key_id` (not the key) in the manifest.

Acceptance: IPFS content is high-entropy (test: `ent` on the blob), a wrong/absent key fails cleanly, and the audit log records `key_id` + `sha256`.

B4. Attack evaluation (3 pd) — makes the privacy claim credible

Build `notebooks/attacks/` and report results — **positive or negative**, both are results:

- **Membership inference** (shadow-model, report AUC/TPR@FPR=1 %), DP on vs off.
- **Model inversion / DLG** on a single site's update, SecAgg on vs off.
- **Gradient leakage** baseline on the current pipeline (expected: trivially successful, which is a great before/after figure for the README).
- **Property inference** on adapter deltas.

Acceptance: `docs/privacy.md` contains a table: attack, setting, metric, before, after. This single table is worth more than ten marketing claims.

---

7. Track C — Blockchain v2 (`FederatedRound.sol`)

Problems being fixed

C-05 (empty submissions accepted) · H-03 (`quorum = 0` deployable) · H-04 (owner = aggregator, zero-address aggregator, no revocation, immortal pause) · H-07 (silent overwrite, removed hospital still counted) · M-10 (docs describe an ABI that doesn't exist).

Design decisions

Decision	Choice	Why
Per-submission tx vs batch	**Events per submission + one Merkle root per round**	Keeps the audit trail, cuts gas ~3×, enables off-chain inclusion proofs
Access control	`AccessControl` roles, not `owner`	Aggregator ≠ governance ≠ pauser
Upgradeability	**UUPS proxy + 48 h timelock** (or immutable v1 for the demo)	Models evolve; immutability is not an excuse to ship a bug
Round finalisation	`seal` → **challenge window** → `finalize`	Gives hospitals a veto window; nothing is "final" the instant the aggregator says so
Stakes	Optional `stake`/`slash`	Needed only if you claim Byzantine resistance
Signature	EIP-712 optional `submitWithSig`	Hospital doesn't need gas; aggregator relays

Core contract (condensed but real)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";
import {Pausable} from "@openzeppelin/contracts/utils/Pausable.sol";
import {MerkleProof} from "@openzeppelin/contracts/utils/cryptography/MerkleProof.sol";

contract FederatedRound is AccessControl, Pausable {
    bytes32 public constant AGGREGATOR_ROLE = keccak256("AGGREGATOR_ROLE");
    bytes32 public constant HOSPITAL_ROLE   = keccak256("HOSPITAL_ROLE");
    bytes32 public constant PAUSER_ROLE     = keccak256("PAUSER_ROLE");

    uint32  public constant MAX_CID_LEN = 128;
    uint256 public immutable challengeWindow;   // seconds

    struct Update {
        bytes32 weightHash;    // sha256(ciphertext adapter)
        bytes32 keyCommitment; // keccak(keyId || salt) — never the key
        uint96  numSamples;
        uint32  lossBps;
        uint64  timestamp;
        bool    slashed;
    }

    enum Phase { None, Open, Sealed, Finalized, Disputed }

    uint256 public currentRound;
    uint256 public quorum;
    mapping(uint256 => Phase) public phase;
    mapping(uint256 => mapping(address => Update)) private _updates;
    mapping(uint256 => address[]) private _submitters;
    mapping(uint256 => bytes32) public merkleRoot;
    mapping(uint256 => uint96)  public totalSamples;
    mapping(uint256 => bytes32) public globalCidHash;
    mapping(uint256 => uint64)  public sealedAt;
    mapping(address => uint256) public stakeOf;
    mapping(address => uint32)  public reputation;

    error BadCid(); error EmptyHash(); error BadQuorum(); error NotOpen();
    error AlreadySubmitted(); error AlreadyRegistered(); error QuorumNotMet(); error NotSealed();
    error WindowOpen(); error WindowClosed(); error AlreadyFinalized();
    error ZeroAddress(); error NoStake();
```

```
  …
    event HospitalRegistered(address indexed hospital, uint96 stake);
    event HospitalRevoked(address indexed hospital, string reason);
    event QuorumChanged(uint256 oldQuorum, uint256 newQuorum);
    event UpdateSubmitted(uint256 indexed round, address indexed hospital,
                          bytes32 weightHash, uint96 numSamples, uint32 lossBps);
    event RoundSealed(uint256 indexed round, bytes32 merkleRoot, uint96 totalSamples, uint32 count);
    event RoundDisputed(uint256 indexed round, address indexed hospital, string reason);
    event RoundFinalized(uint256 indexed round, bytes32 globalCidHash, string globalCid);
```

```
    event Slashed(address indexed hospital, uint256 amount, string reason);

    constructor(uint256 quorum_, uint256 challengeWindow_) {
        if (quorum_ == 0) revert BadQuorum();
        quorum = quorum_;
        challengeWindow = challengeWindow_;
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(AGGREGATOR_ROLE, msg.sender);
        _grantRole(PAUSER_ROLE, msg.sender);
        currentRound = 1;
        phase[1] = Phase.Open;
    }

    // ---- governance ----
    function registerHospital(address h, uint96 stake) external onlyRole(DEFAULT_ADMIN_ROLE) {
        if (h == address(0)) revert ZeroAddress();
        if (hasRole(HOSPITAL_ROLE, h)) revert AlreadyRegistered();
        stakeOf[h] = stake;
        _grantRole(HOSPITAL_ROLE, h);
        emit HospitalRegistered(h, stake);
    }

    function revokeHospital(address h, string calldata reason) external onlyRole(DEFAULT_ADMIN_ROLE) {
        _revokeRole(HOSPITAL_ROLE, h);          // AND remove from round accounting off-chain
        emit HospitalRevoked(h, reason);
    }

    function setQuorum(uint256 q) external onlyRole(DEFAULT_ADMIN_ROLE) {
        if (q == 0) revert BadQuorum();
        emit QuorumChanged(quorum, q);
        quorum = q;                              // note: effective next round, documented
    }

    address public aggregator;
```

```
…
  function setAggregator(address a) external onlyRole(DEFAULT_ADMIN_ROLE) {
      if (a == address(0)) revert ZeroAddress();
      address old = aggregator;
      if (old != address(0)) _revokeRole(AGGREGATOR_ROLE, old);
      aggregator = a;
      _grantRole(AGGREGATOR_ROLE, a);
      emit AggregatorChanged(old, a);
  }

  // ---- hospital ----
  function submitUpdate(
      uint256 round,
      string calldata cid,
      bytes32 weightHash,
      bytes32 keyCommitment,
      uint96 numSamples,
      uint32 lossBps
  ) external onlyRole(HOSPITAL_ROLE) whenNotPaused {
      if (phase[round] != Phase.Open) revert NotOpen();
      if (bytes(cid).length == 0 || bytes(cid).length > MAX_CID_LEN) revert BadCid();
      if (weightHash == bytes32(0)) revert EmptyHash();
      if (numSamples == 0) revert BadQuorum();
      if (_updates[round][msg.sender].timestamp != 0) revert AlreadySubmitted();

      _updates[round][msg.sender] = Update(weightHash, keyCommitment, numSamples, lossBps, uint64(block.timestamp), false);
      _submitters[round].push(msg.sender);
      totalSamples[round] += numSamples;
      reputation[msg.sender] += 1;
      emit UpdateSubmitted(round, msg.sender, weightHash, numSamples, lossBps);
  }

  // ---- aggregator ----
  function sealRound(uint256 round, bytes32 root, bytes32 cidHash)
      external onlyRole(AGGREGATOR_ROLE) whenNotPaused
  {
      if (phase[round] != Phase.Open) revert NotOpen();
      if (_submitters[round].length < quorum) revert QuorumNotMet();
      if (root == bytes32(0) || cidHash == bytes32(0)) revert EmptyHash();
      merkleRoot[round] = root;
      globalCidHash[round] = cidHash;
```

```
        sealedAt[round] = uint64(block.timestamp);
        phase[round] = Phase.Sealed;
        emit RoundSealed(round, root, totalSamples[round], uint32(_submitters[round].length));
    }
```

```
  …
    function dispute(uint256 round, string calldata reason) external onlyRole(HOSPITAL_ROLE) {
        if (phase[round] != Phase.Sealed) revert NotSealed();
        if (block.timestamp > sealedAt[round] + challengeWindow) revert WindowClosed();
        phase[round] = Phase.Disputed;
        emit RoundDisputed(round, msg.sender, reason);
    }

    function finalizeRound(uint256 round, string calldata globalCid)
        external onlyRole(AGGREGATOR_ROLE)
    {
        if (phase[round] != Phase.Sealed) revert NotSealed();
        if (block.timestamp < sealedAt[round] + challengeWindow) revert WindowOpen();
        phase[round] = Phase.Finalized;          // never re-open, never overwrite
        emit RoundFinalized(round, globalCidHash[round], globalCid);
        currentRound = round + 1;
        phase[round + 1] = Phase.Open;
    }

    // ---- verification ----
    function verifyInclusion(uint256 round, bytes32 leaf, bytes32[] calldata proof) external view returns (bool) {
        return MerkleProof.verify(proof, merkleRoot[round], leaf);
    }

    function leafOf(uint256 round, address h) public view returns (bytes32) {
        Update memory u = _updates[round][h];
        return keccak256(abi.encode(round, h, u.weightHash, u.keyCommitment, u.numSamples, u.lossBps));
    }
}
```

> The `AggregatorChanged` event must also be declared with the other events (`event AggregatorChanged(address indexed oldAggregator, address indexed newAggregator);`). The audit probe P4 exists precisely because the current contract forgot the zero-check **and** the event.

Tests you must add (Foundry)

Test	Assertion
`test_RevertWhen_EmptyCid`	`submitUpdate("" , …)` reverts `BadCid`
`testFuzz_QuorumZero`	constructor always reverts for `q == 0`
`test_RevertWhen_DoubleSubmit`	second submission in same round reverts
`test_RevertWhen_SealBelowQuorum`	seal with `count < quorum` reverts
`test_RoundIsImmutable`	`finalizeRound` twice reverts; `merkleRoot` unchanged
`test_DisputeBlocksFinalize`	disputed round cannot be finalised
`test_RevokedHospitalCannotSubmit`	role revoked → reverts `Not hospital`
`test_ZeroAggregator`	`setAggregator(address(0))` reverts
`invariant_NoRoundAdvancesWithoutQuorum`	Foundry invariant across random call sequences
`test_GasSubmitUnder150k`	`assertLt(gasUsed, 150_000)`

Plus `slither .` clean, `foundry coverage` ≥ 95 %.

Acceptance: all ten above pass; `docs/abi.md` is generated from artefacts (no hand-written ABI drift), and the README's Solidity snippets are extracted from the actual files (see §19 anti-patterns).

---

8. Track D — Storage v2

Change	Detail	pd
Private swarm	Generate `swarm.key`, mount identically into all 4 Kubo containers, `LIBP2P_FORCE_P2PSTREAM=true`; verifies isolation (no public DHT)	1
IPFS Cluster	Add `ipfs-cluster` with `replication_factor: 2-3`; pinning becomes a cluster operation, not a per-node `ipfs add`	2
Correct client	multipart `/api/v0/add?pin=true&cid-version=1&raw-leaves=true`; verify returned CID against local multihash; `raise` on mismatch	1
CAR export	`/api/v0/dag/export` per round → `artifacts/rounds/NNNN/bundle.car` for offline audit	1
Encryption	B3 before add; store `key_id` + `sha256(ciphertext)`	(B3)
Retrieval with verification	fetch → `sha256` → compare on-chain `weightHash` → decrypt → load; fail closed	1
Retention	pin TTL per round (e.g. keep 10 rounds); `ipfs pin rm` + garbage collect; documented	0.5
Health	`/api/v0/version`, `/api/v0/id`, cluster `/id`; **any non-200 or empty body = unhealthy** (the current emulator returns 200 + empty body for unknown routes — never trust that)	0.5

---

9. Track E — Orchestration, API, and the database

E1. FastAPI service (4 pd)

Replace `SimpleHTTPRequestHandler` with FastAPI + SQLAlchemy + Alembic. Why: you need validation, auth, async I/O, OpenAPI docs, and testability — all things the stdlib server makes painful.

```
POST   /auth/login                      → JWT (role claim)
GET    /api/status                      → real aggregate (see below)
GET    /api/hospitals                   → DB + chain
GET    /api/hospitals/{id}              → detail, contribution history
GET    /api/rounds                      → paginated
GET    /api/rounds/{id}                 → manifest + metrics + submissions
GET    /api/rounds/{id}/submissions/{site} → cid, sha256, tx, inclusion proof
POST   /api/rounds                       → start round (admin)
POST   /api/rounds/{id}/submit           → hospital upload (mTLS or JWT+hospital role)
POST   /api/rounds/{id}/seal            → aggregator
POST   /api/rounds/{id}/finalize        → aggregator
POST   /api/diagnose                     → real model, calibrated, with evidence
GET    /api/audit/events                 → indexed chain events
GET    /api/privacy/ledger               → ε per site per round
POST   /api/admin/hospitals/{id}/revoke  → governance
GET    /healthz /readyz /metrics         → probes + Prometheus
WS     /ws/rounds                        → live state push
```

`/api/status` must tell the truth about provenance:

```
{
  "generated_at": "2026-10-22T09:31:04Z",
  "sources": {"chain": "ok", "ipfs": "degraded", "model": "ok", "db": "ok"},
  "round": {"id": 4, "phase": "finalized", "participants": 3, "required": 3,
            "started_at": "...", "finalized_at": "...", "tx_hash": "0x…", "merkle_root": "0x…"},
  "model": {"version": "v4.0.4", "cid": "bafy…", "sha256": "0x…", "params": 295424,
            "metrics": {"macro_f1": 0.51, "top5": 0.88, "ece": 0.071}},
  "privacy": {"dp": true, "epsilon_total": 7.8, "delta": 1e-05, "secagg": true},
  "hospitals": [{"id": "h1", "name": "…", "status": "online", "last_seen": "…", "epsilon": 2.4}]
}
```

If `ipfs: degraded`, the UI must render a banner. Never substitute stale numbers silently — that was finding M-01.

E2. Round state machine (2 pd)

```
# src/medxchain/rounds/state.py
class Phase(StrEnum):
    IDLE = "idle"; COLLECTING = "collecting"; QUORUM = "quorum_reached"
    SEALING = "sealing"; SEALED = "sealed"; AGGREGATING = "aggregating"
    PUBLISHING = "publishing"; FINALIZED = "finalized"; DISPUTED = "disputed"
    FAILED = "failed"; ABORTED = "aborted"

ALLOWED = {
    Phase.IDLE: {Phase.COLLECTING},
    Phase.COLLECTING: {Phase.QUORUM, Phase.ABORTED, Phase.FAILED},
    Phase.QUORUM: {Phase.SEALING, Phase.ABORTED},
    Phase.SEALING: {Phase.AGGREGATING, Phase.FAILED},
    Phase.AGGREGATING: {Phase.PUBLISHING, Phase.FAILED},
    Phase.PUBLISHING: {Phase.SEALED, Phase.FAILED},
    Phase.SEALED: {Phase.FINALIZED, Phase.DISPUTED},
    Phase.DISPUTED: {Phase.FAILED, Phase.SEALED},
    Phase.FINALIZED: {Phase.IDLE},
}

def transition(rnd, to: Phase) -> None:
    if to not in ALLOWED[rnd.phase]:
        raise IllegalTransition(f"{rnd.phase} -> {to}")
    rnd.phase = to            # persisted with SELECT ... FOR UPDATE
```

Persist with `SELECT ... FOR UPDATE` (optimistic locking), so two aggregator workers can't double-seal. Add an **idempotency key** on every POST and a `round_events` append-only table — the audit trail lives there, not only on-chain.

E3. Worker + queue (2 pd)

Celery/RQ with Redis is enough. Jobs: `train_local` (edge, not server), `aggregate`, `publish_ipfs`, `anchor_chain`, `evaluate`, `index_events`. Requirements: retries with exponential backoff, dead-letter queue, per-round timeout, and a visible job status in the UI. Kill the current "synchronously do everything in one script with `try/except: fake it`" model.

E4. Chain client + event indexer (2 pd)

```
# src/medxchain/chain/client.py
class ChainClient:
    def __init__(self, rpc: str, address: str, abi_path: Path, key: str | None = None):
        self.w3 = Web3(Web3.HTTPProvider(rpc, request_kwargs={"timeout": 10}))
        self.contract = self.w3.eth.contract(address=Web3.to_checksum_address(address), abi=json.loads(abi_path.read_text()))
        self.account = self.w3.eth.account.from_key(key) if key else None

    def submit(self, round_id: int, cid: str, weight_hash: bytes, key_commitment: bytes, n: int, loss_bps: int) -> str:
        fn = self.contract.functions.submitUpdate(round_id, cid, weight_hash, key_commitment, n, loss_bps)
        tx = fn.build_transaction({"from": self.account.address, "nonce": self.w3.eth.get_transaction_count(self.account.address),
                                   "gas": 250_000, "gasPrice": self.w3.eth.gas_price})
        signed = self.account.sign_transaction(tx)
        return self.w3.eth.send_raw_transaction(signed.raw_transaction).hex()
```

Indexer: poll `UpdateSubmitted`/`RoundSealed`/`RoundFinalized` → upsert into `chain_events`; expose via `/api/audit/events`. For production, use The Graph or Ponder; for local, the poller is fine. Verify `deployment_info.json`'s address matches `contract.address` on start — 13 hard-coded addresses is 13 future bugs.

---

10. Track F — Dashboard 2.0

F1. Data binding (3 pd) — closes M-01

Replace every literal with API-driven rendering:

Element	Today	Target
Hospitals connected	literal `14`	`len(status.hospitals)`
Model updates	literal `1.2k`	`SELECT count(*) FROM submissions`
Global version	derived from round, but only badge	`status.model.version` + CID + sha
Accuracy chart	hard-coded coordinate array	`/api/rounds?fields=metrics` series
Participation bars	literal percentages	per-site `num_samples / total`
Ledger rows	2023-10-24 + `0xdeadbeef…`	`/api/audit/events`
SHAP bars	hard-coded divs	attributions from `/api/diagnose` (Track G)
"124 hospitals / 1.2M gradients / 99.9 %"	literals	delete or compute

Add: skeleton loaders, explicit empty states, and an **error banner with retry** when a source is degraded. Add `data-testid` attributes so Playwright can assert no element ever shows a hard-coded number.

F2. Auth + RBAC (3 pd)

JWT with roles → view access:

Role	Sees	Can do
`clinician`	diagnosis, anamnesis	request diagnosis, override, comment
`hospital_op`	own site only	upload update, rotate keys, view ε
`auditor`	everything read-only, replay, exports	verify inclusion proofs, export
`admin`	all	register/revoke hospitals, set quorum, pause
`public`	none (optional read-only transparency page)	—

Implementation notes: `HttpOnly` cookie or `Authorization: Bearer`; short TTL + refresh; `Depends(require_roles("auditor"))`; every mutation writes an `audit_log` row with `actor, action, target, before, after, ip, trace_id`. Never ship the current "anyone can POST `/api/advance-round`" behaviour.

F3. Accessibility & polish (2 pd)

Today: **0** `aria-*`, **0** `role=`, emoji-as-icons, no focus management, fonts from Google (breaks offline previews). Target WCAG 2.1 AA:

- semantic landmarks (`nav`, `main`, `aside`), `aria-current` on tabs, `role="tablist"/"tab"/"tabpanel"` with arrow-key handling;
- visible focus rings, `prefers-reduced-motion`, contrast ≥ 4.5:1;
- real SVG icons inline (no emoji), every icon `aria-hidden` with a text label;
- self-host Inter/JetBrains Mono (`woff2` in `web/public/fonts`);
- `axe-core` in CI on every view; Playwright keyboard-only smoke test.

F4. Frontend build (2 pd)

Move to Vite + ES modules (or React if the team prefers). Why: you now have ~8 views, real auth, WebSocket, and shared components. Keep it framework-light if you want (`vite` + vanilla + small `lit`/`htm` templates), but get: hashing for cache-busting, tree-shaking, a single `npm run build`, and a CSP-friendly bundle. Replace hand-rolled canvas charts with `uPlot` (tiny, fast) or `Chart.js`; feed them API arrays.

---

11. Track G — Clinical AI that is safe to show a doctor

Current state: substring matching on disease names. It returned **"Open Wound Of The Chest 49 %"** for dyspnoea. This track is not optional if the demo is shown to anyone clinical.

G1. Real inference service (2 pd)

```
# src/medxchain/clinical/infer.py
class DiagnosisService:
    def __init__(self, ckpt_dir: Path, device="cpu"):
        self.ckpt = load_checkpoint(ckpt_dir / "global.pt")
        self.model = load_model_from_checkpoint(self.ckpt, device=device).eval()
        self.tok = AutoTokenizer.from_pretrained(self.ckpt.base_model)

    @torch.inference_mode()
    def predict(self, symptoms: str, top_k: int = 5) -> Prediction:
        enc = self.tok(symptoms, return_tensors="pt", truncation=True, max_length=128).to(self.device)
        logits = self.model(enc["input_ids"], enc["attention_mask"])
        probs = torch.softmax(self.temperature(logits), dim=-1)[0]
        return Prediction(probs=probs, top_k=top_k, model_version=self.ckpt.round, cid=self.ckpt_cid)
```

Guardrails (all cheap, all important):

- reject `< 3` symptoms or `< 10` chars with a 422 + guidance;
- **red-flag detection** ("chest pain", "suicidal", "stroke", "severe bleeding") → return an urgent-care advisory before any differential;
- cap `top_k` at 5, always show the model version, round, CID, and "decision support only";
- log the redacted query (strip anything that looks like a name/MRN) with a trace id;
- rate-limit per user/IP.

G2. Calibration (1.5 pd)

```
def fit_temperature(logits, labels):           # on a held-out calibration split
    T = nn.Parameter(torch.ones(1))
    opt = torch.optim.LBFGS([T], lr=0.1, max_iter=50)
    def closure():
        opt.zero_grad(); loss = F.cross_entropy(logits / T, labels); loss.backward(); return loss
    opt.step(closure); return float(T.detach())
```

Report **ECE** before/after. A model whose confidence is meaningless is worse than no confidence bar: the current UI paints 49 % on nonsense.

G3. Conformal prediction + abstention (2 pd)

```
# split-conformal, per class, on a calibration set
def conformal_threshold(scores_correct: np.ndarray, alpha=0.1) -> float:
    n = len(scores_correct)
    q = min(1.0, np.ceil((n + 1) * (1 - alpha)) / n)
    return float(np.quantile(scores_correct, q))

def prediction_set(probs: torch.Tensor, thr: float, k_max: int = 5) -> list[tuple[int, float]]:
    idx = (probs >= thr).nonzero(as_tuple=True)[0].tolist()
    return sorted(((int(i), float(probs[i])) for i in idx), key=lambda t: -t[1])[:k_max]
```

Display: **"Prediction set (90 % coverage): 3 conditions"** instead of fake precision, and **abstain** (`"insufficient evidence — recommend workup X"`) when the set is empty or larger than `k_max`. This is the single most defensible clinical feature you can add in a semester, and it's easy to evaluate (coverage vs set size plot).

G4. Explainability that is real (2 pd)

Replace the fake SHAP bars with token-level attribution that matches a text model:

- **Integrated gradients / attention rollout** over input tokens → highlight the symptoms that drove the prediction;
- or **LIME** (`lime` is already in `requirements.txt` and unused) on the text — slow but honest;

- **similar-case retrieval**: kNN over MedCPT embeddings of the training set, show 3 anonymised neighbours as evidence;
- if you insist on SHAP, use `shap.Explainer` on the classifier head and label it "feature attribution", not "consensus across hospitals".

Add an **evidence pack** export (feature F8): one PDF per diagnosis with model version, round, CID, tx hash, calibration, prediction set, top attributions, retrieved neighbours, and a disclaimer.

G5. ICD-10 mapping (2 pd)

The 793 free-text labels need codes for any real integration. Build `data/icd10_map.csv` (label → ICD-10-CM → chapter), validate coverage (target ≥ 95 % of label mass), and display both. This also makes the ontology auditable and lets you group rare classes into chapters for macro metrics.

---

12. Track H — 18 new features

Grouped, each with why / how / effort / priority / depends. These are the things that turn the project from "course assignment" into "product with a story".

Trust & audit

F1. Round Replay (time-travel debugger) · P1 · 3 pd

- **Why:** an auditor must be able to reconstruct any round without trusting you; it is also the best debugging tool you will ever build.
- **How:** `GET /api/rounds/{id}/replay` returns manifest + submissions + merkle proof + metrics + chain events; UI scrubs through phases with a timeline; `medxchain round replay 4 --verify` re-checks every hash/CID and prints PASS/FAIL per artefact.
- **Acceptance:** replay of round 4 on a fresh machine reproduces `metrics.json` within tolerance and re-verifies all CIDs.

F2. Privacy (ε) ledger · P1 · 2 pd

- **Why:** makes the privacy claim measurable and auditable; it is what a DPIA needs.
- **How:** table `epsilon_ledger(round, site, epsilon, delta, noise_mult, clip, accountant)`; per-site and cumulative budgets; UI gauge with a budget bar per hospital + CSV export; refuse to start a round that would exceed the configured budget.
- **Acceptance:** budget exhaustion blocks a round with a clear error; export matches the accountant's numbers.

F3. Inclusion-proof verifier · P2 · 1.5 pd

- **Why:** lets a hospital prove its update was counted — the actual point of the Merkle root.
- **How:** `/api/rounds/{id}/submissions/{site}/proof` → `{leaf, proof[], root}`; UI "Verify on-chain" button calls `verifyInclusion` via RPC and shows ✓/✗; public verify page (no auth) taking round+address.
- **Acceptance:** tampering with any leaf makes verification fail.

F4. Immutable audit export · P2 · 1.5 pd

- **Why:** regulators, journals, and your own future self need a portable artefact.
- **How:** `GET /api/audit/export?round=4&format=pdf|json|csv` → signed bundle (`sha256` + optional Ed25519 signature), including chain events, ε, metrics, disclaimers; `docs/compliance.md` explains how to verify offline.
- **Acceptance:** exported bundle re-verifies with `medxchain audit verify bundle.zip`.

Privacy & governance

F5. Consent & purpose registry · P2 · 3 pd

- **Why:** GDPR/HIPAA need purpose limitation and demonstrable consent, not just "no raw data leaves".
- **How:** store `consent(patient_pseudonym_hmac, purpose, granted_at, revoked_at, policy_version)`; on-chain store only `keccak(policy_version)`; training refuses sites whose consent coverage for the purpose < threshold; dashboard shows coverage per purpose.
- **Acceptance:** revoking consent excludes the site from the next round and the manifest records why.

F6. Crypto-shredding / right-to-erasure flow · P2 · 2 pd

- **Why:** resolves the immutability-vs-GDPR-Art-17 tension without mutating the chain.
- **How:** per-round key in DB/KMS; `DELETE /api/privacy/keys/{key_id}` destroys the key → ciphertext becomes unreadable; on-chain hash remains but is no longer linkable; audit log records the erasure event and the affected rounds.
- **Acceptance:** after key destruction, `/rounds/{id}/artifact` returns `410 Gone` with an erasure reference; inclusion proof still verifies.

F7. Hospital onboarding + key ceremony · P2 · 3 pd

- **Why:** a real deployment's weakest link is how a new hospital joins.
- **How:** wizard: generate Ed25519 + X25519 keys → CSRs → admin approval → `registerHospital` tx → stake → mTLS cert issued → connectivity self-test (chain RPC, IPFS, API) → first dry-run round. Store key fingerprints; support rotation with overlap.
- **Acceptance:** a new site can join end-to-end in < 15 min without touching the codebase.

Clinical

F8. Evidence pack PDF per diagnosis · P1 · 2 pd

- **Why:** clinicians don't trust a percentage; they trust a traceable artefact.
- **How:** WeasyPrint/ReportLab template: input (redacted), prediction set + coverage, calibration, top attributions, 3 similar cases, model version/round/CID/tx, disclaimer; QR code linking to the public verify page (F3).
- **Acceptance:** PDF contains the tx hash and re-verifies.

F9. Human-in-the-loop review queue · P2 · 3 pd

- **Why:** captures clinical feedback (the only ground truth you'll ever get) and creates an active-learning loop.
- **How:** `POST /api/diagnose/{id}/review {agree, corrected_label, note}`; queue view for clinicians; safe aggregation of corrections into a separate federated "feedback" round (never silently into the diagnostic model); track agreement rate and per-class disagreement.
- **Acceptance:** overrides are stored with actor + timestamp, appear in audit export, and never mutate past round artefacts.

F10. Drift & data-quality monitor · P2 · 3 pd

- **Why:** a federated model silently rots when one site's population shifts.
- **How:** per site per round: label-distribution JS divergence vs baseline, input-length and vocabulary drift, missing-field rate, PSI on numeric features; alert when above threshold; store the series for charts.
- **Acceptance:** injecting a synthetic 30 % label shift triggers an alert in the next round.

F11. Champion / challenger · P3 · 3 pd

- **Why:** lets you ship a new aggregation algorithm safely.
- **How:** two model versions scored on the same shadow set each round; promotion gate (`macro_f1 >= champion + 0.01` and `ece <= champion`), manual approval, rollback by round id.

- **Acceptance:** a deliberately worse challenger is never promoted.

Operations

F12. Federated cohort discovery · P3 · 4 pd

- **Why:** the most compelling "why federated" demo — "which sites have ≥ 20 cases of this rare condition?" without exposing records.
- **How:** each site returns a noisy count per query label (Laplace/Gaussian, ε = 0.1); aggregator sums; UI shows counts with confidence intervals and suppresses cells < k (k-anonymity, e.g. 5).
- **Acceptance:** reconstructed counts within CI; no cell below k is ever displayed.

F13. Simulation sandbox / digital twin · P2 · 4 pd

- **Why:** "what if we add a 4th hospital / drop quorum to 2 / turn DP on?" — answers with numbers instead of opinions.
- **How:** run the aggregator against synthetic sites with configurable α (non-IID), dropout rate, and attack injectors; output a comparison table + charts; no chain writes.
- **Acceptance:** reproduces the FedAvg vs FedProx gap on a synthetic non-IID split.

F14. Alert center + incident workflow · P2 · 2 pd

- **Why:** an ops product needs to tell you when it's broken before a demo does.
- **How:** rules engine (round stuck > N min, host unreachable, ε budget > 80 %, hash mismatch, chain indexer lag, drift); dedupe/ack/silence; Slack/email webhook; each alert links to the affected round.
- **Acceptance:** killing a Kubo container raises `ipfs.unhealthy` within one scrape interval and clears on recovery.

Platform

F15. Python hospital SDK + CLI · P2 · 3 pd

- **Why:** a hospital should not clone your repo to participate.
- **How:** `pip install medxchain-client`; `Client(rpc, ipfs, key).round(4).train(csv).submit()`; `medxchain doctor` runs preflight (chain, IPFS, cert, clock skew, disk); typed errors; offline dry-run mode.
- **Acceptance:** a 20-line script trains, encrypts, uploads, submits, and returns the tx hash.

F16. Benchmark leaderboard · P2 · 2 pd

- **Why:** turns "we built a system" into "we measured algorithms", which is what a paper needs.
- **How:** `bench/` runs FedAvg / FedProx / SCAFFOLD / trimmed-mean × IID / Dirichlet(0.5) / Dirichlet(0.1) × DP on/off; stores results; renders a sortable table with mean ± std over 3 seeds.
- **Acceptance:** one command produces the table in `results/leaderboard.md`.

F17. Public transparency page · P3 · 1.5 pd

- **Why:** the civic argument for this project — prove the network is healthy without exposing anything.
- **How:** unauthenticated read-only page: number of sites (opt-in), rounds completed, aggregate metrics, ε policy, incident history. **No** CIDs, no addresses, no PHI, no per-site numbers.
- **Acceptance:** security review confirms zero identifying data.

F18. PWA + offline clinician mode · P3 · 2 pd

- **Why:** hospitals have bad connectivity; the diagnosis view is the one that must work.
- **How:** service worker caches the shell + model; on-device inference via ONNX-exported head (LoRA-merged adapter is small: ~1 MB int8); queue diagnoses, sync when online; never cache other patients' data.
- **Acceptance:** airplane-mode diagnosis works and syncs idempotently.

---

13. Track I — Security engineering

Write `docs/threat-model.md` (STRIDE table) before writing code. Priority items:

Area	Control	pd
AuthN	JWT + refresh, short TTL, `jti` revocation list; hospital auth via **mTLS**	2
AuthZ	role checks on every route (deny-by-default dependency), object-level checks (a site only sees itself)	1
Transport	TLS everywhere (`mkcert` in dev, real certs in staging); HSTS	1
Input	Pydantic models, size caps (`max_body=1 MB`, uploads `50 MB`), regex on CIDs, reject unknown fields	1
Secrets	`.env` never committed; SOPS/Vault; **Anvil keys are dev-only** and must be refused if `env=prod`	0.5
Keys	Ed25519 signing, X25519 wrapping, rotation, fingerprints, HSM/KMS for prod	2
Supply chain	`pip-audit`, `npm audit --omit=dev`, `trivy fs`, SBOM (`syft`), Dependabot, pinned hashes	1
Contracts	`slither`, `foundry fmt`, invariant tests, timelock, multisig (Safe 2-of-3) as admin	3
Abuse	rate limiting (`slowapi`), request ids, per-IP quotas on `/diagnose`	1
Web	CSP (no inline), `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`, CSRF token for cookie auth	1
Logging	structured, PII-redacted, `trace_id`, no secrets, 90-day retention policy	1
Backups	Postgres PITR, IPFS pin snapshots, restore drill documented	1

Also: **rotate every key currently in the repo** (Anvil account #0 private key is public knowledge; `.env.example` has a stale deploy address). Treat the committed `deployment_info.json` as compromised-by-assumption.

---

14. Track J — Compliance & governance

You cannot say "HIPAA compliant" — you can say "designed against HIPAA Security Rule §164.312 controls; not yet audited". Build the evidence:

Artefact	Contents	pd
`docs/compliance/hipaa-mapping.md`	Each safeguard → implementation → test/evidence (access control, audit logs, integrity, transmission security)	2
`docs/compliance/gdpr-dpia.md`	Processing purposes, lawful basis, data minimisation, DPIA with DP + SecAgg + crypto-shredding, retention, sub-processors	2
`docs/compliance/kvkk.md`	Turkish law counterpart (the team context)	1
`docs/compliance/data-retention.md`	Round artefacts 10 rounds, logs 90 days, keys per policy, deletion procedure	0.5
`docs/compliance/samd-statement.md`	Explicit "research use only; not a medical device; no clinical claims" until validated	0.5
Consent + erasure	Features F5, F6	(tracked)
Data processing register	Table of datasets, owners, locations, legal basis	0.5

Governance (if you keep the "decentralized" framing): admin = **Safe multisig 2-of-3** (two institutions + one neutral), all role changes behind a 48 h timelock, and an ADR explaining why a single EOA was rejected. Otherwise, rename the project "federated learning with an on-chain audit log" — which is honest and still interesting.

---

15. Track K — MLOps, CI/CD, observability, deployment

K1. CI (2 pd)

```
# .github/workflows/ci.yml (jobs)
lint:      ruff check . && ruff format --check . && mypy src
py-test:   pytest -q --cov=medxchain --cov-fail-under=70
contracts: forge fmt --check && forge test -vvv && forge coverage --report summary
security:  slither contracts/ && pip-audit && npm audit --omit=dev
web:       npm ci && npm run lint && npm run build && npx playwright test
e2e:       docker compose up -d --wait && pytest -m e2e
```

Nightly: full e2e + benchmark smoke + dependency update PR. Pre-commit: `ruff`, `mypy`, `prettier`, `solhint`, `gitleaks`, `check-added-large-files` (would have caught the 67 MB CSVs).

K2. Observability (2 pd)

- **Metrics** (Prometheus): `round_duration_seconds{phase}`, `submissions_received`, `quorum_wait_seconds`, `ipfs_upload_bytes`, `ipfs_latency_seconds`, `chain_tx_gas_used`, `chain_indexer_lag_blocks`, `epsilon_total{site}`, `model_macro_f1`, `diagnose_requests_total{outcome}`.
- **Traces** (OpenTelemetry): one trace per round, spans per hop (train → encrypt → ipfs → chain → aggregate → evaluate).
- **Logs**: JSON, `trace_id`/`round_id`/`site_id`, no PII.
- **Dashboards**: Grafana JSON committed in `ops/grafana/`; alerts (F14) wired to the same rules.
- **SLOs** in §16.

K3. Deployment (2 pd)

Multi-stage Dockerfile (`python:3.12-slim`, non-root, healthcheck), full `docker-compose.yml` (anvil, 4 kubo, ipfs-cluster, postgres, redis, api, worker, web, prometheus, grafana, loki), `Makefile` targets (`make up`, `make seed`, `make round`, `make bench`), and a Helm chart for k8s. One command must bring up the whole system — the current README requires six terminals and manual address copying.

```
# docker-compose.yml (excerpt)
services:
  anvil:   { image: ghcr.io/foundry-rs/foundry:latest, command: anvil --host 0.0.0.0 --chain-id 31337, ports: ["8545:8545"] }
  ipfs1:   { image: ipfs/kubo:latest, volumes: ["./ops/swarm.key:/data/ipfs/swarm.key", "ipfs1:/data/ipfs"], ports: ["5001:5001"] }
  cluster: { image: ipfs/ipfs-cluster:latest, depends_on: [ipfs1, ipfs2, ipfs3], ports: ["9094:9094"] }
  db:      { image: postgres:16, environment: { POSTGRES_DB: medxchain }, volumes: ["pg:/var/lib/postgresql/data"] }
  api:     { build: ., command: uvicorn medxchain.api.app:app --host 0.0.0.0 --port 8000, depends_on: [db, anvil] }
  worker:  { build: ., command: celery -A medxchain.rounds.workers worker -Q rounds }
  web:     { build: ./web, ports: ["3000:80"], environment: { API_URL: "http://api:8000" } }
```

---

16. Targets & SLOs

Metric	Today	Target	How measured
Round wall-clock (3 sites, LoRA)	n/a	< 90 s local	`round_duration_seconds`
Submissions per round	0 (fake)	3/3 with proof	chain events
Upload size per site	3.2 MB toy / 440 MB real	≤ 2 MB (LoRA fp32) / 0.35 MB int8	IPFS size
Gas per submission	~200 k	≤ 150 k	`forge snapshot`
Dashboard truthfulness	2/12 panels live	12/12 live, 0 literals	Playwright assertion
Test coverage	0 % Python	≥ 70 % Python, ≥ 95 % contracts	coverage reports
Macro-F1 (global)	73.9 % acc (unknown F1)	≥ 78 % acc, ≥ 0.55 macro-F1	`metrics.json`
Calibration ECE	not measured	≤ 0.08	calibration report
Conformal coverage (α=0.1)	n/a	0.88–0.92	calibration report
ε per round	n/a	≤ 2.0; total ≤ 8	Opacus accountant

Metric	Today	Target	How measured
MIA AUC	presumably ~1.0	≤ 0.55 at ε ≤ 8	attack notebook
API p95 latency	n/a	< 300 ms (non-ML routes)	Prometheus
Uptime (dev compose)	n/a	99 % over a week	probes

---

17. Finding → work-item coverage

Every audit finding must be closed by something in this plan:

Finding	Closed by
C-01 no real training	A1, A3
C-02 fabricated metrics	§1.1, A5, E1, F1
C-03 PII in Git	§1.4, J (retention), history scrub
C-04 unsafe diagnosis	§1.3, §1.10, G1–G4
C-05 empty submissions	C (contract v2)
H-01 chain never called	E4, A6
H-02 broken IPFS + fake CIDs	§1.7, D
H-03 quorum 0	C
H-04 admin centralization	C (roles, multisig, timelock)
H-05 unauthenticated API	E1, F2, I
H-06 incompatible checkpoints	A2
H-07 silent overwrite	C (immutable finalize)
H-08 federation hurts accuracy	A2 (LoRA), A4 (FedProx), L (benchmarks)
M-01 mockup dashboard	F1
M-02 contract sprawl	C, §1.11
M-03 no CI/licence/tests	§1.9, §1.12, K1
M-04 repo hygiene	§1.5, §1.6, §1.11, K1
M-05 reproducibility	A3, A6, K3
M-06 privacy claims	B1–B4, J
M-07 backend robustness	E1–E3
M-08 data quality	A1, A5, G5
M-09 silent failures	A3, E3, F14
M-10 docs/ABI drift	C, §19
L-*	F3, K1, I

---

18. The first 10 pull requests (copy-paste backlog)

1. **PR-01 "honesty pass"** — remove fake metrics, stub `/diagnose` to 501, add disclaimers, add LICENSE, fix gitignores. (1 pd)
2. **PR-02 "contract safety"** — three `require`s + `setAggregator` zero-check + tests for each. (1 pd)
3. **PR-03 "repo slimming"** — dedupe CSVs, strip PII columns, drop `__pycache__`, move notebooks. (1 pd)
4. **PR-04 "packaging + CI"** — `pyproject.toml`, `src/medxchain`, ruff/mypy/pytest, GitHub Actions. (2 pd)

5. **PR-05 "real training"** — `data.py` + `trainer.py` + seeded deterministic test. (3 pd)
6. **PR-06 "checkpoint + LoRA"** — one artefact format, `peft` integration, param-count test. (2 pd)
7. **PR-07 "chain v2"** — `FederatedRound.sol` + Foundry fuzz/invariant suite + `web3.py` client + one real tx from the pipeline. (4 pd)
8. **PR-08 "IPFS correctness"** — multipart, CID verify, encryption, retrieval fail-closed. (2 pd)
9. **PR-09 "service + DB"** — FastAPI skeleton, Postgres schema, `/api/status` from real data, round state machine. (4 pd)
10. **PR-10 "dashboard truth"** — bind every panel to the API, add error states, Playwright literal-detection test. (3 pd)

After PR-10 the project is honest and closed-loop. Everything else is depth.

---

19. What NOT to build (yet)

- **Tokenomics / governance token.** Adds legal surface, zero scientific value; a multisig is enough.
- **On-chain model storage.** Prohibitively expensive, and you'd be publishing weights — the opposite of the privacy pitch.
- **Full zkML proofs of training.** Research-grade cost (minutes–hours per proof for tiny models). If you want ZK, start with a **norm-bound range proof** on the update (proves clipping, i.e. DP is honest) — that's tractable with Bulletproofs.
- **Kubernetes on day one.** Docker Compose until you have a second environment.
- **Microservices.** One API + one worker; split later if you ever need to.
- **A custom blockchain.** Anvil→Base/Sepolia is fine; the interesting work is the protocol, not consensus.
- **"HIPAA-compliant" wording.** Until an audit happens, say "designed against". Overclaiming is the repo's existing failure mode.
- **Frontend framework rewrite for its own sake.** A build step plus a chart library is enough.

---

20. Definition of done (per track)

A track ships only when **all** of these are true:

- [ ] Code lives under `src/medxchain` with types (`mypy --strict` on new modules)
- [ ] Unit + integration tests; coverage gate respected
- [ ] Failure paths tested (dependency down, hash mismatch, hash of empty, timeout)
- [ ] No literal appears in the UI that isn't in a test fixture
- [ ] Metrics/traces emitted; alert rule exists for the new failure mode
- [ ] `docs/` updated; README claim added **only** with a link to the artefact proving it
- [ ] Security review: inputs validated, authz checked, secrets externalised
- [ ] Migration + rollback documented (DB and contract)
- [ ] Manifest fields added so the round remains replayable

---

21. Risks & trade-offs

Risk	Mitigation
LoRA-only training caps accuracy vs full fine-tuning	Report both; full-model mode behind a flag for offline runs
DP-SGD on CPU is slow	Ghost clipping + small adapter + small batch accumulation; GPU optional
SecAgg is complex and easy to get subtly wrong	Start with synchronous rounds + dropout exclusion; add recovery later; property-test the sum
Challenge window slows the demo	`challengeWindow = 60` in dev, `1 day` in prod; **document** the difference
More infra → more ways to fail during a demo	`make demo` runs a fully local, seeded, offline path; record a backup video
Scope explosion	The phase table is the contract: do not start Phase 3 before Phase 1 exits
Privacy claims outrun implementation	Every claim in README must cite `docs/privacy.md` numbers

---

22. Appendix A — environment variables

```
MEDXCHAIN_ENV=dev
MEDXCHAIN_DEMO_MODE=true            # blocks fake data outside dev
MEDXCHAIN_CHAIN_RPC=http://127.0.0.1:8545
MEDXCHAIN_CHAIN_ID=31337
MEDXCHAIN_CONTRACT=0x…              # never hard-code; loaded from deployment_info.json by CLI
MEDXCHAIN_AGGREGATOR_KEY=0x…        # dev only; refuses to start when ENV=prod
MEDXCHAIN_IPFS_API=http://127.0.0.1:5001
MEDXCHAIN_IPFS_CLUSTER=http://127.0.0.1:9094
MEDXCHAIN_DB_URL=postgresql+psycopg://medxchain:medxchain@db:5432/medxchain
MEDXCHAIN_REDIS_URL=redis://redis:6379/0
MEDXCHAIN_JWT_SECRET=…
MEDXCHAIN_QUORUM=3
MEDXCHAIN_DP_ENABLED=true
MEDXCHAIN_DP_TARGET_EPSILON=8
MEDXCHAIN_DP_DELTA=1e-5
MEDXCHAIN_DP_MAX_GRAD_NORM=1.0
MEDXCHAIN_SECAGG_ENABLED=true
MEDXCHAIN_MODEL_BASE=ncbi/MedCPT-Query-Encoder
MEDXCHAIN_MODEL_TUNABLE=lora
MEDXCHAIN_CHALLENGE_WINDOW=60
MEDXCHAIN_LOG_LEVEL=info
```

Appendix B — database schema (essentials)

```
CREATE TABLE hospitals (id UUID PRIMARY KEY, name TEXT NOT NULL, address TEXT UNIQUE NOT NULL,
                        public_key TEXT, status TEXT NOT NULL DEFAULT 'pending', created_at TIMESTAMPTZ DEFAULT now());
CREATE TABLE rounds (id BIGINT PRIMARY KEY, phase TEXT NOT NULL, quorum INT NOT NULL,
                     merkle_root BYTEA, global_cid TEXT, global_sha256 BYTEA, tx_hash TEXT,
                     started_at TIMESTAMPTZ, sealed_at TIMESTAMPTZ, finalized_at TIMESTAMPTZ);
CREATE TABLE submissions (round_id BIGINT REFERENCES rounds(id), hospital_id UUID REFERENCES hospitals(id),
                      cid TEXT NOT NULL, ciphertext_sha256 BYTEA NOT NULL, key_id TEXT NOT NULL,
                      num_samples INT NOT NULL CHECK (num_samples > 0), loss_bps INT,
                      epsilon REAL, delta REAL, tx_hash TEXT, created_at TIMESTAMPTZ DEFAULT now(),
                      PRIMARY KEY (round_id, hospital_id));
CREATE TABLE metrics (round_id BIGINT, scope TEXT, entity_id TEXT, name TEXT, value DOUBLE PRECISION,
                      PRIMARY KEY (round_id, scope, entity_id, name));
CREATE TABLE epsilon_ledger (round_id BIGINT, hospital_id UUID, epsilon REAL NOT NULL, delta REAL NOT NULL,
                       noise_mult REAL, clip REAL, accountant TEXT, created_at TIMESTAMPTZ DEFAULT now(),
                       PRIMARY KEY (round_id, hospital_id));
CREATE TABLE chain_events (id BIGSERIAL PRIMARY KEY, block_number BIGINT, tx_hash TEXT, log_index INT,
                       event TEXT, round_id BIGINT, payload JSONB, observed_at TIMESTAMPTZ DEFAULT now(),
                       UNIQUE (tx_hash, log_index));
CREATE TABLE audit_log (id BIGSERIAL PRIMARY KEY, actor TEXT, role TEXT, action TEXT, target TEXT,
                       before JSONB, after JSONB, ip INET, trace_id TEXT, created_at TIMESTAMPTZ DEFAULT now());
CREATE TABLE consent (id BIGSERIAL PRIMARY KEY, patient_hmac BYTEA, purpose TEXT, policy_version TEXT,
                       granted_at TIMESTAMPTZ, revoked_at TIMESTAMPTZ);
CREATE TABLE alerts (id BIGSERIAL PRIMARY KEY, rule TEXT, severity TEXT, payload JSONB, round_id BIGINT,
                       created_at TIMESTAMPTZ DEFAULT now(), acked_by TEXT, acked_at TIMESTAMPTZ);
```

Appendix C — dependency additions

```
# pyproject.toml
dependencies = [
  "fastapi>=0.115", "uvicorn[standard]>=0.30", "pydantic>=2.8", "pydantic-settings>=2.4",
  "sqlalchemy>=2.0", "psycopg[binary]>=3.2", "alembic>=1.13", "redis>=5", "celery>=5.4",
  "web3>=7", "httpx>=0.27", "cryptography>=43",
  "torch>=2.3", "transformers>=4.44", "peft>=0.12", "opacus>=1.5",
  "scikit-learn>=1.5", "pandas>=2.2", "numpy>=1.26", "scipy>=1.13",
  "structlog>=24", "prometheus-client>=0.20", "opentelemetry-sdk>=1.26",
  "weasyprint>=62", "pyjwt>=2.9", "slowapi>=0.1.9",
]
dev = ["pytest", "pytest-cov", "pytest-asyncio", "hypothesis", "ruff", "mypy", "types-requests", "pip-audit"]
```

Contract tooling: `foundry` (forge/cast/anvil), `@openzeppelin/contracts@5`, `slither-analyzer`.

---

23. TL;DR

1. **Stop the lying first** (12 quick wins, 1 week) — fake accuracy, fake CIDs, fake transactions, unsafe diagnosis, PII in Git.
2. **Close the loop** (weeks 2–4) — real training on real rows, LoRA adapters, a hardened contract that actually receives transactions, Postgres as the source of truth, a dashboard bound to the API.
3. **Make the privacy real** (weeks 5–6) — DP-SGD with a published ε, secure aggregation, encrypted blobs, and an attack-evaluation table.
4. **Make the clinical AI defensible** (weeks 7–8) — calibration, conformal prediction sets, abstention, real explanations, ICD-10.
5. **Then add product features** (weeks 9–12) — round replay, ε ledger, inclusion proofs, evidence packs, onboarding, drift, champion/challenger, cohort discovery, benchmarks.
6. **Ship evidence, not adjectives** — every README claim links to a metrics file, a test, or a compliance artefact.

Appendix A — Task backlog summary

The full machine-readable version — acceptance criteria, verify commands, file targets, stop conditions and regression guards — is `agent_backlog.yaml` in the repository root. This is the human-readable summary; the YAML is the contract.

ID	P	Task	Closes	pd	Depends
T-01	P0	Remove fabricated metrics from the status API	C-02	0.5	—
T-02	P0	Harden ModelUpdateTracker (empty CID, quorum 0, zero aggregator)	C-05, H-03, H-04	1	—
T-03	P0	Stop serving substring-matching diagnosis	C-04	0.5	—
T-04	P0	Strip PII columns from committed datasets	C-03, M-04	1	—
T-05	P0	Deduplicate repo assets, fix ignore rules	M-04	0.5	—
T-06	P0	Correct IPFS client: multipart, verify, no invented CIDs	H-02	1	—
T-07	P0	Packaging, linting, CI skeleton	M-03	2	—
T-08	P1	Real local training, seeded, metric-producing	C-01, M-05, M-09	3	T-07
T-09	P1	Single checkpoint format + LoRA adapters	H-06, H-08	2	T-08
T-10	P1	Weighted FedAvg + robust aggregators + property tests	M-01, H-08	2	T-09
T-11	P1	FederatedRound.sol v2: roles, Merkle root, challenge window	C-05, H-04, H-07, M-10	4	T-02
T-12	P1	Python chain client + event indexer; pipeline really submits	H-01	3	T-11
T-13	P1	FastAPI service, Postgres schema, round state machine	H-05, M-07	4	T-07
T-14	P1	Bind the dashboard to real data; zero literals	M-01	3	T-13
T-15	P1	Real diagnosis with calibration, conformal sets, abstention	C-04, M-06	3	T-09, T-13
T-16	P2	DP-SGD over adapters + ε ledger	M-06	3	T-09, T-13
T-17	P2	Encrypted artefacts before IPFS + crypto-shredding	M-06	2	T-06, T-12
T-18	P2	Round manifest, replay, signed audit export	M-05, M-09	3	T-12, T-13

Total: 37 person-days. P0 (T-01…T-07, 6.5 pd) makes the project honest; P1 (T-08…T-15, 24 pd) closes the loop; P2 (T-16…T-18, 8 pd) makes it credible. Start at T-01 and do not skip a gate.